

Умножение больших чисел. Возведение в степень и разложение на простые множители

Павел Атнашев
patnashev@gmail.com

6 сентября 2023 г.

Введение

С тех пор, как был изобретён натуральный счёт, люди начали сталкиваться с таким явлением, как большие числа. Для упрощения операций с ними во многих языках возникли специальные слова, обозначающие числа больше десяти: «дюжина», «сотня», «тысяча». Когда этого стало мало, появились такие слова как «миллион», «миллиард»/«биллион», «триллион». Для больших чисел оказалось удобнее использовать степень десятки. В одном грамме вещества находится около 10^{24} нуклонов (число Авогадро). В наблюдаемой части Вселенной около 10^{80} нуклонов (число Эддингтона).

Может показаться, что числа, превышающие количество материи во Вселенной, бесполезны. Однако они возникают в различных математических дисциплинах и бывают полезны на практике, например, в криптографии. Современная криптография оперирует числами от 100 до 1000 десятичных цифр. Несмотря на внушительные размеры таких чисел, современные вычислительные устройства с лёгкостью справляются с основными математическими операциями над ними.

Но человеческое любопытство не знает границ. В таких дисциплинах, как экспериментальная математика и вычислительная теория чисел, люди ставят вопросы и находят ответы о свойствах чисел из миллионов десятичных цифр. На момент написания этих строк, самое большое известное простое число состояло из 24 862 048 цифр. Благодаря специально разработанным алгоритмам, арифметические операции над такими числами на современных компьютерах выполняются за несколько миллисекунд.

Тем не менее, математике известны нетривиальные числа, размер которых не позволяет нам производить даже простейшие арифметические операции над ними. Наибольшее известное число Кармайкла состоит из 295 486 761 787 цифр. Наибольшее число Ферма, у которого известен делитель, состоит из $10^{5488968}$ цифр. Всех атомов во Вселенной не хватит, чтобы даже просто записать это число в десятичном виде. Но мы знаем, что оно делится на вполне осязаемое простое число.

В данной работе в качестве «больших» рассматриваются числа от 1 тысячи до 100 миллионов десятичных цифр. Для записи таких чисел требуется от нескольких килобайт до нескольких десятков мегабайт оперативной памяти или дискового пространства. Операции сложения и вычитания линейны и не представляют сложности в большинстве случаев. Основные трудности возникают с операциями умножения и умножения по модулю. Именно эти операции мы рассматриваем в первой главе. Вторая глава посвящена возведению больших чисел в большие степени, а также вопросам проверки корректности вычислений. В третьей главе мы исследуем возможность разложения больших чисел на простые множители.

Оглавление

1	Умножение	3
1.1	Представление	3
1.2	Умножение	4
1.3	Быстрое умножение	5
1.4	Преобразование Фурье	7
1.5	Комплексное поле	8
1.6	Конечное поле	10
1.7	Иерархическая память	11
1.8	Умножение по модулю	12
1.9	Взвешенное преобразование	14
1.10	Свободное пространство слов	16
1.11	Вложенное умножение	18
1.12	Реализация	20
2	Возведение в степень	22
2.1	Двоичная степень	22
2.2	Проверка Гербица	23
2.3	Сертификат Атнашева	25
2.4	Функция Петшака	28
2.5	Небинарная степень	29
2.6	Обобщение Ли	31
2.7	Последовательности Люка	33
2.8	Эллиптические кривые Эдвардса	36
3	Разложение на простые	41

Глава 1

Умножение

1.1 Представление

Высокопроизводительные вычислительные устройства работают с типами данных, имеющими фиксированный размер. Для достижения максимальной производительности операций, большие числа должны быть представлены в виде множества отдельных элементов, размер которых оптимален для конкретного вычислительного устройства. Современные центральные процессоры имеют специализированные блоки, способные исполнять арифметические операции сразу над несколькими числами с плавающей запятой, благодаря чему такой тип данных является предпочтительным. Напротив, графические (параллельные) процессоры оптимизированы под выполнение операций над потоками коротких целочисленных данных. Для оптимальных результатов на таких процессорах нужны соответствующие алгоритмы. Такая ситуация приводит к тому, что для решения одной и той же задачи могут использоваться совершенно разные представления и разные алгоритмы, в зависимости от доступного вычислительного устройства.

Удобнее всего большое число представлять в виде многочлена над некоторым полем $\mathbb{F}$, принимающего значение равное данному большому числу в фиксированной точке, называемой «основанием» (*base*). Обозначим большое число как $\mathbf{a}$, соответствующий ему многочлен как $A(x)$, его коэффициенты как a_i , его степень как $N - 1$. Тогда

$$\mathbf{a} = A(\text{base}) = \sum_{i=0}^{N-1} a_i \text{base}^i, \quad a_i \in \mathbb{F}, \quad \text{base} \in \mathbb{F} \quad (1.1.1)$$

Примером такого представления является десятичная система счисления. В ней используется основание 10, а у каждого возможного значения коэффициента есть свой символ, называемый «цифрой».

$$3196 = [3x^3 + 1x^2 + 9x + 6](10) = 3 \cdot 1000 + 1 \cdot 100 + 9 \cdot 10 + 6$$

Стоит заметить, что такое представление не зависит от того, в какой системе счисления мы работаем. В шестнадцатеричной системе счисления представление числа C7C по основанию A всё равно будет 3 1 9 6:

$$\text{C7C}_{16} = [3x^3 + 1x^2 + 9x + 6](A_{16}) = 3_{16} \cdot \text{E8}_{16} + 1_{16} \cdot 64_{16} + 9_{16} \cdot A_{16} + 6_{16}$$

Для больших оснований, коэффициенты многочлена часто называют «словами», реже «цифрами» (не нужно путать с десятичными цифрами). В отличие от систем счисления, на слова не накладываются строгие ограничения вроде $0 \leq a_i < \text{base}$. Существуют алгоритмы, эффективно работающие с отрицательными словами, иррациональными словами, словами, превышающими по модулю квадрат основания. Единственное требование: $A(\text{base}) = \mathbf{a}$. Очевидно, что при таких условиях представление любого числа не уникально. Этот факт используется во многих алгоритмах умножения, в стадии так

называемой «нормализации», когда одно представление заменяется на другое, более удобное для дальнейших операций.

На двоичных вычислительных системах, коими являются абсолютное большинство современных устройств, логично в качестве основания выбирать степень двойки. Это делает хранение больших чисел более эффективным, ускоряет построение представления и упрощает нормализацию. Но в некоторых ситуациях такое основание является не самым лучшим выбором. Например, при операциях по модулю $b^n + c$ бывает выгодно в качестве основания взять b или даже $\sqrt{b}$. Построение такого представления становится нетривиальной, но решаемой задачей.

1.2 Умножение

Пусть $\mathbf{a}$ и $\mathbf{b}$ два больших числа, $A(x)$ и $B(x)$ соответствующие многочлены. Произведение $\mathbf{a} \cdot \mathbf{b}$ может быть вычислено как произведение двух многочленов:

$$P(x) = A(x) \times B(x) \quad (1.2.1)$$

$$\mathbf{a} \cdot \mathbf{b} = P(base) \quad (1.2.2)$$

Наиболее простым способом вычисления $P(x)$ является хорошо известный всем с начальной школы метод умножения столбиком:

$$p_k = \sum_{i+j=k} a_i b_j \quad (1.2.3)$$

Этот метод не только прост, но и весьма эффективен. Для небольших N он оказывается быстрее более сложных методов за счёт отсутствия накладных расходов. Кроме того, его можно использовать для отладки других методов даже для больших N . Вычисление одного отдельного p_k выполняется за линейное время и может служить для «контроля качества» более сложного алгоритма умножения. Для такой проверки выбирается случайное k , и сравниваются вычисленные разными методами p_k .

Важной задачей является оценка абсолютной величины p_k , с целью подбора правильного типа данных для хранения этих коэффициентов. При одинаковой степени исходных многочленов, коэффициент p_{N-1} будет равен

$$p_{N-1} = a_0 b_{N-1} + a_1 b_{N-2} + \dots + a_{N-2} b_1 + a_{N-1} b_0,$$

его абсолютная величина может достигать $N \cdot base^2$, при условии, что a_i и b_j меньше $base$. Но если $P(x)$ будет использован в последующей операции умножения, то коэффициенты результата будут ещё больше. Чтобы не допустить бесконтрольного роста коэффициентов, используется операция «нормализации», или приведения коэффициентов к нормальному для заданного представления виду. Эта операция также известна как «распространение переноса» (carry propagation).

Допустим, норма представления задана как $0 \leq a_i < base$, тогда нормализацию можно описать как нахождение нового многочлена $P^{norm}(x)$ такого, что

$$P^{norm}(base) = P(base), 0 \leq p_k^{norm} < base \quad (1.2.4)$$

Важно понимать, что многочлен $P^{norm}(x)$ не равен многочлену $P(x)$, у них даже могут быть разные степени. Но их значения равны в точке $base$. Таким образом, операция нормализации завязана на конкретное значение $base$, и наличие этой операции отличает умножение больших чисел от умножения многочленов.

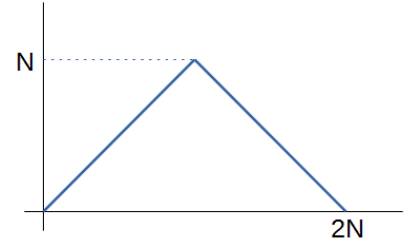


Рис. 1.1: Количество слагаемых в p_k .

Также, в стадии нормализации удобно выполнять небольшие дополнительные операции. Например, умножение на маленькую константу вроде 2 или -1, прибавление маленькой константы к результату и подобные.

Конкретная реализация нормализации сильно зависит от представления, но в целом она напоминает школьное сложение с переносом в следующий разряд. К каждому коэффициенту последовательно прибавляется перенос, результат сравнивается с основанием, вычисляется новый текущий коэффициент, и формируется следующий перенос. Рассмотрим пример. В ходе возведения в квадрат числа 3196 в десятичном представлении с помощью (1.2.3) получается следующий многочлен:

$$P(x) = 9x^6 + 6x^5 + 55x^4 + 54x^3 + 93x^2 + 108x + 36$$

После нормализации получается многочлен большей степени:

$$P^{norm}(x) = 1x^7 + 0x^6 + 2x^5 + 1x^4 + 4x^3 + 4x^2 + 1x + 6$$

Но оба многочлена имеют одинаковое значение в точке 10:

$$P^{norm}(10) = P(10) = 10214416$$

1.3 Быстрое умножение

Алгоритм умножения столбиком имеет сложность $O(N^2)$. Долгое время считалось, что такая сложность теоретически оптимальна, пока Анатолий Карацуба не предложил свой метод. Карацуба стал своего рода Лобачевским в области вычислительной сложности. После того, как он сломал «квадратичную стену», было найдено несколько ещё более быстрых алгоритмов.

Метод Карацубы является рекурсивным алгоритмом, на каждом шаге которого каждый множитель разбивается на две части. Но вместо четырёх умножений выполняется три, плюс несколько дополнительных сложений и вычитаний.

$$\{a, b\} \times \{c, d\} = \{ac, ad + bc, bd\} = \{ac, (a + b)(c + d) - ac - bd, bd\} \quad (1.3.1)$$

Сложность этого алгоритма равна $O(N^{\log_2 3})$, $\log_2 3 = 1,58\dots$

Существуют алгоритмы со сложностью $O(N \log N \log \log N)$. В определённом смысле, их можно считать линейными. Именно благодаря этим алгоритмам возможен поиск простых чисел из десятков миллионов цифр. Общепринято называть такие алгоритмы «быстрыми».

Быстрые алгоритмы основываются на следующем свойстве многочленов:

$$\forall r \in \mathbb{F}: P(r) = A(r) \cdot B(r) \quad (1.3.2)$$

Вместо вычисления коэффициентов многочлена $P(x)$, вычисляется его значение в некотором множестве различных точек $\{r_i\}$. Для этого вычисляются значения многочленов $A(x)$ и $B(x)$ в этих точках, а затем перемножаются. Коэффициенты многочлена $P(x)$ могут быть восстановлены с помощью ещё одного свойства многочленов, известного как теорема Безу:

$$P(r) = P(x) \bmod (x - r) \quad (1.3.3)$$

Напомним, что для многочленов операция взятия остатка по модулю легко реализуется с помощью деления столбиком.

Таким образом, мы можем объявить $\{P(r_i)\}$ множеством многочленов нулевой степени, каждый из которых является остатком многочлена $P(x)$ по одному из взаимно простых модулей $x - r_i$. Для

нахождения остатков по модулям бóльших степеней, мы можем последовательно применять Китайскую теорему об остатках, которая для произвольных многочленов $R(x)$ и $Q(x)$ формулируется как

$$P(x) \bmod (R(x) \times Q(x)) = ((P(x) \bmod R(x)) \times Q(x)^{-1} \bmod R(x)) \times Q(x) + ((P(x) \bmod Q(x)) \times R(x)^{-1} \bmod Q(x)) \times R(x) \quad (1.3.4)$$

пока не останется только один многочлен

$$P_N(x) = P(x) \bmod \prod_{i=0}^{N-1} x - r_i \quad (1.3.5)$$

Степень этого многочлена будет не выше $N-1$. Выбрав N таким образом, что $N > \deg(A(x)) + \deg(B(x))$, мы можем быть уверены, что $P_N(x) = P(x)$.

Для оптимальной производительности, (1.3.4) следует применять к парам многочленов одинаковой степени. В результате, на первом шаге мы получим $N/2$ многочленов степени 1, на втором шаге $N/4$ многочленов степени 3, и так далее, пока на $\log N$ шаге не получим один многочлен степени $N-1$.

Также, не следует забывать, что (1.3.4) содержит в себе умножение многочленов, что делает этот алгоритм рекурсивным. Его сложность можно оценить как $O(N \log^2 N)$, что несколько хуже указанной ранее. Но мы можем улучшить этот алгоритм.

Дело в том, что мы ничем не ограничены в выборе множества точек $\{r_i\}$. Мы вольны выбрать такие точки, какие нам будут удобны в дальнейших вычислениях. Наиболее популярным выбором являются корни многочлена $x^N - 1$ (рис. 1.2). Рассмотрим случай, когда N является степенью двойки. У такого множества есть очень полезное свойство: для любой точки r , множеству также принадлежат $-r$ и r^2 . Отсортируем точки таким образом, чтобы на каждом шаге точки r и $-r$ стояли рядом:

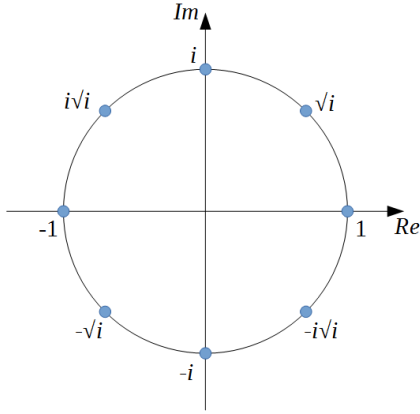


Рис. 1.2: Корни многочлена $x^8 - 1$.

$$\{r_i\} = 1, -1, i, -i, \sqrt{i}, -\sqrt{i}, i\sqrt{i}, -i\sqrt{i}, \dots \quad (1.3.6)$$

Что даёт нам двоичное дерево соответствующих многочленов:

$$\begin{aligned} (x-1) \times (x+1) &= x^2 - 1 \\ (x-i) \times (x+i) &= x^2 + 1, \quad (x^2-1) \times (x^2+1) = x^4 - 1 \\ (x-\sqrt{i}) \times (x+\sqrt{i}) &= x^2 - i \\ (x-i\sqrt{i}) \times (x+i\sqrt{i}) &= x^2 + i, \quad (x^2-i) \times (x^2+i) = x^4 + 1, \quad (x^4-1) \times (x^4+1) = x^8 - 1 \end{aligned}$$

Легко видеть, что на каждом шаге мы получаем многочлены вида $x^n - r$, где n является степенью двойки, а r является одним из первых N/n элементов (1.3.6). Используем это в (1.3.4), приняв $R(x) = x^n - r$, $Q(x) = x^n + r$, $P(x) \bmod R(x) = P^+(x)$, $P(x) \bmod Q(x) = P^-(x)$:

$$\begin{aligned} P(x) \bmod (x^{2n} - r^2) &= (P^+(x) \times (x^n + r)^{-1} \bmod (x^n - r)) \times (x^n + r) \\ &\quad + (P^-(x) \times (x^n - r)^{-1} \bmod (x^n + r)) \times (x^n - r) \\ &= (P^+(x) \cdot \frac{1}{2r}) \times (x^n + r) + (P^-(x) \cdot \frac{-1}{2r}) \times (x^n - r) \\ &= \frac{P^+(x) + P^-(x)}{2} + \frac{P^+(x) - P^-(x)}{2r} \times x^n \end{aligned} \quad (1.3.7)$$

В программной реализации этого алгоритма удобно хранить коэффициенты $P^+(x)$ и $P^-(x)$ в одном массиве, так что коэффициенты $P^-(x)$ выглядят как p_{n+k} . В таком случае (1.3.7) превращается в:

$$p'_k = \frac{p_k + p_{n+k}}{2}, \quad p'_{n+k} = \frac{p_k - p_{n+k}}{2r} \quad (1.3.8)$$

Обычно $\frac{1}{2}$ выносят за скобки на каждом шаге и в конце домножают результат на $\frac{1}{N}$.

Благодаря удачному выбору множества $\{r_i\}$, мы свели рекурсивное умножение многочленов к линейной операции, состоящей из сложения, вычитания и умножения на обратное к корню. То же самое можно сделать и с вычислением значений многочленов $A(x)$ и $B(x)$ в точках $\{r_i\}$. Для этого используется следующее свойство модульной операции:

$$A(x) \bmod R(x) = A(x) \bmod (R(x) \times Q(x)) \bmod R(x) \quad (1.3.9)$$

В двоичном дереве многочленов любой узел делит всех своих предков, а значит для вычисления остатка по модулю узла можно использовать остаток по модулю любого предка. В начале, многочлен $A(x)$ принимается равным $A(x) \bmod (x^N - 1)$, после чего вычисляются многочлены по модулям $x^{N/2} - 1$ и $x^{N/2} + 1$. Мы можем это сделать, поскольку эти модули делят $x^N - 1$. Эта операция повторяется рекурсивно, уменьшая каждый раз степень многочленов в два раза, пока по теореме Безу мы не получаем значения $A(x)$ в точках, равных корням $x^N - 1$.

На каждом шаге рекурсии мы имеем многочлены вида $A(x) \bmod (x^{2^n} - r^2)$. Нашей задачей является получить многочлены $A^+(x)$ и $A^-(x)$ такие, что

$$A^+(x) = A(x) \bmod (x^n - r), \quad A^-(x) = A(x) \bmod (x^n + r) \quad (1.3.10)$$

Опять же, удобно хранить коэффициенты $A^+(x)$ и $A^-(x)$ в одном массиве, коэффициенты $A^-(x)$ как a'_{n+j} . Тогда, выполнив деление столбиком, мы получим:

$$a'_j = a_j + a_{n+j}r, \quad a'_{n+j} = a_j - a_{n+j}r \quad (1.3.11)$$

Эта операция так же состоит из одного умножения, сложения и вычитания. На следующем шаге рекурсии для $A^+(x)$ используются многочлены $x^{n/2} - \sqrt{r}$ и $x^{n/2} + \sqrt{r}$, произведение которых равно $x^n - r$. Для $A^-(x)$, соответственно, многочлены $x^{n/2} - i\sqrt{r}$ и $x^{n/2} + i\sqrt{r}$. В результате, на каждом шаге мы используем многочлены со свободными членами, являющимися первыми N/n элементами последовательности (1.3.6). Более того, подобную рекурсию можно использовать для собственно построения последовательности (1.3.6).

Легко видеть, что операции (1.3.11) и (1.3.8) являются обратными друг другу.

1.4 Преобразование Фурье

Искушённый читатель мог заметить, что операции (1.3.11) и (1.3.8) также известны как прямое и обратное быстрое преобразование Фурье (БПФ/FFT). Это действительно так, и нетрудно понять почему.

БПФ является быстрым алгоритмом вычисления дискретного преобразования Фурье (ДПФ/DFT), которое определяется как

$$\bar{a}_k = \sum_{j=0}^{N-1} a_j e^{-i \frac{2\pi}{N} jk} \quad (1.4.1)$$

Здесь и далее черта над символом означает преобразованные данные. Легко заметить, что каждый коэффициент a_j умножается на одну и ту же величину в степени j . Это является тождественным вычислению значения многочлена $A(x)$ в этой точке:

$$\bar{a}_k = A(e^{-i \frac{2\pi}{N} k}) \quad (1.4.2)$$

Точки $e^{-i \frac{2\pi}{N} k}$ в комплексной плоскости равномерно расположены на единичной окружности. Каждая из них в степени N равна 1. Иными словами, эти точки являются корнями уравнения $x^N - 1 = 0$. Получается, что дискретное преобразование Фурье по определению является вычислением значений многочлена $A(x)$ в корнях многочлена $x^N - 1$ — именно то, что мы и делали в предыдущем разделе.

В практическом плане это означает, что все быстрые алгоритмы, разработанные для вычисления ДПФ, могут быть использованы и для умножения больших чисел. В том числе алгоритмы с N неравным степени двойки.

В то же время, нет необходимости хорошо разбираться в свойствах преобразования Фурье, чтобы эффективно перемножать большие числа. Более того, это может быть вредным, поскольку сужает область восприятия только до вопросов, связанных с ДПФ. Но это не единственное возможное преобразование. Например, на практике часто используют в качестве множества $\{r_i\}$ корни многочлена $x^N + 1$ (рис. 1.3). Алгоритм остаётся таким же быстрым, но он уже не является преобразованием Фурье. Попытки объяснить его в терминах дискретных преобразований могут привести к ненужной путанице и ложному ощущению, что это какой-то совершенно другой алгоритм.

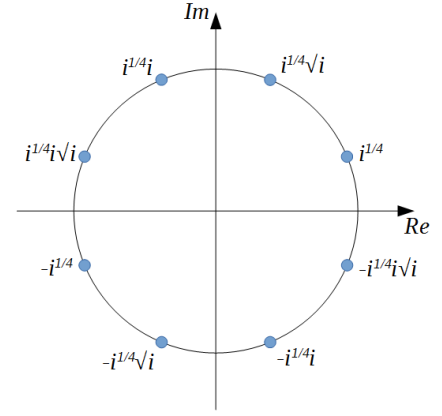


Рис. 1.3: Корни многочлена $x^8 + 1$.

Ситуация усугубляется тем, что этот алгоритм не имеет даже собственного названия. На практике используются названия «отрицательная половина БПФ», «негациклическая свёртка» (negacyclic convolution), «z-преобразование с $z^N = -1$ », «чирп-алгоритм» (chirp z-transform), «взвешенное преобразование с весом $\sqrt[N]{-1}$ », «вычисление значений многочлена в корнях $x^N + 1$ ». И, что самое печальное, по историческим причинам некоторые библиотеки, реализующие этот алгоритм, продолжают называть его «БПФ», хотя он таковым не является. И это можно понять, поскольку различия с точки зрения программирования невелики.

Алгоритм реализуется аналогично БПФ, операции (1.3.11) и (1.3.8) не меняются, меняется только выбор r на каждом следующем шаге рекурсии. Домножая каждую точку на $\sqrt[N]{-1}$ («поворачивая» на половину угла), мы получаем следующее дерево многочленов:

$$\begin{aligned} (x - \sqrt[8]{-1}) \times (x + \sqrt[8]{-1}) &= x^2 - \sqrt{i} \\ (x - \sqrt[8]{-1}i) \times (x + \sqrt[8]{-1}i) &= x^2 + \sqrt{i}, \quad (x^2 - \sqrt{i}) \times (x^2 + \sqrt{i}) = x^4 - i \\ (x - \sqrt[8]{-1}\sqrt{i}) \times (x + \sqrt[8]{-1}\sqrt{i}) &= x^2 - i\sqrt{i} \\ (x - \sqrt[8]{-1}i\sqrt{i}) \times (x + \sqrt[8]{-1}i\sqrt{i}) &= x^2 + i\sqrt{i}, \quad (x^2 - i\sqrt{i}) \times (x^2 + i\sqrt{i}) = x^4 + i, \quad (x^4 - i) \times (x^4 + i) = x^8 + 1 \end{aligned}$$

Следует отметить, что используемые корни также используются и в БПФ в два раза большей длины, в данном случае длины 16. Именно поэтому возникло название «отрицательная половина БПФ». Но в отличие от БПФ, каждый корень используется только один раз во всём дереве многочленов.

1.5 Комплексное поле

Наиболее очевидным выбором поля для коэффициентов является поле комплексных чисел $\mathbb{F} = \mathbb{C}$, реализуемое с помощью чисел с плавающей запятой с двойной точностью (double, FP64). Операции с таким типом данных могут выполняться весьма эффективно благодаря поддержке современными центральными процессорами векторных инструкций (SSE, AVX, AVX-512), позволяющих выполнять одну операцию сразу над 8-ю числами, а также дополнительных инструкций вроде умножения-сложения с однократным округлением (fused multiply-add, FMA).

Так как производительность операций зависит прежде всего от их количества, при выборе представления большого числа стараются обеспечить как можно меньшее N и, соответственно, как можно большее основание. Как уже было упомянуто, ограничением тут является способность типа данных хранить коэффициенты результата. В переменной типа FP64 можно хранить целые числа до 2^{53} без потери младших разрядов. В случае, если хотя бы один коэффициент результата превышает этот предел, его значение становится неточным, а результат в целом неверным. Такая ситуация называется ошибкой

округления (roundoff error). Вопросам, связанным с ошибками округления, приходится уделять очень много внимания на практике.

Конечно, можно оценить худший возможный случай, и выбрать N достаточно большое, чтобы гарантированно не выйти за ограничения типа данных. Несмотря на необходимость производить больше операций, такой подход оправдан там, где требуется абсолютная гарантия корректности вычислений. Например, при быстрой проверке результатов работы какого-нибудь алгоритма. Для основной же массы вычислений желателен более тонкий подход.

Основная сложность, связанная с ошибками округления, состоит в том, что они не очень хорошо изучены с теоретической стороны. Многие из нижесказанного было получено эмпирическим путём.

Для оценки уровня ошибки, мы будем использовать максимальное отклонение значения коэффициентов от целой величины. Очевидно, что оно не может превышать 0,5, иначе мы просто не сможем отличить одно целое значение от другого. Кроме того, желательно установить некоторый предельно допустимый уровень S , достижение которого сигнализировало бы, что мы опасно приблизились к зоне ошибок. Часто в качестве такого уровня берут 0,4.

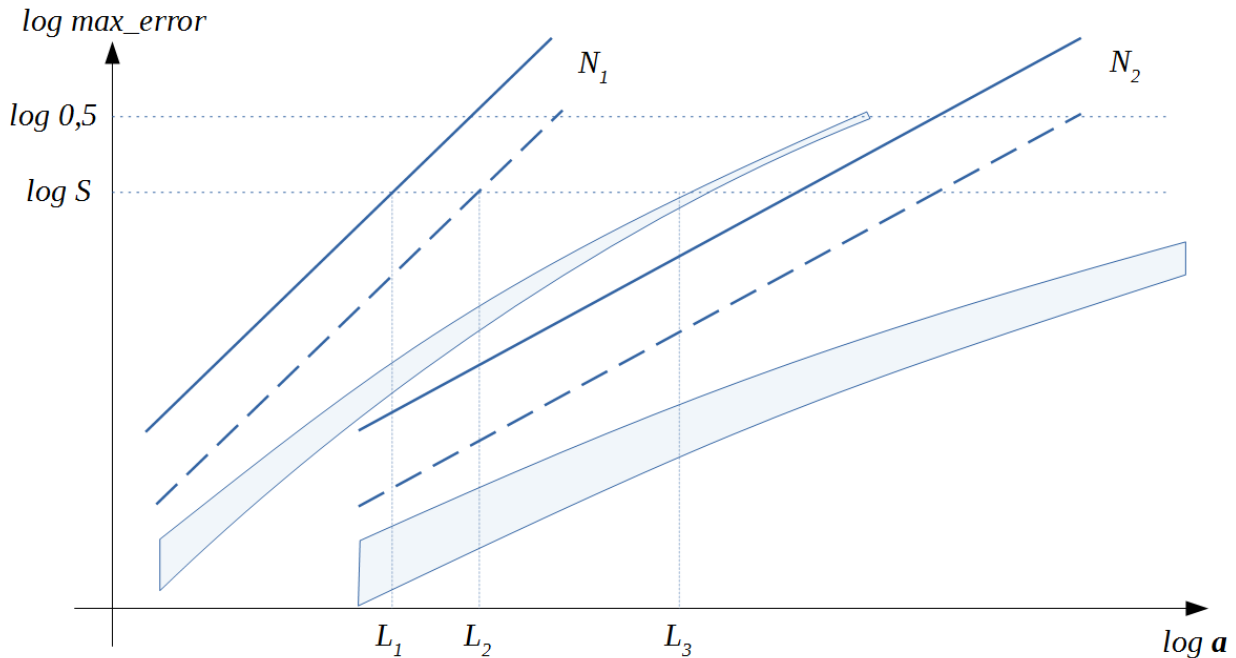


Рис. 1.4: Ошибка округления для двух преобразований длины N_1 и N_2 . Сплошная линия соответствует теоретически максимальному значению. Пунктирная линия соответствует практически наблюдаемому максимальному уровню для “несчастливых” чисел. Закрашенная область соответствует диапазону ошибок для случайных чисел.

На рис. 1.4 схематически представлены уровни ошибок при умножении больших чисел разной длины используя два фиксированных преобразования. При увеличении размера числа, а значит и увеличения основания представления, ошибки нарастают. В какой-то момент использование преобразования длины N_1 становится опасным, уровень ошибок достигает S . В этот момент следует переключиться на преобразование длиной N_2 . Но есть несколько стратегий выбора момента, когда это следует сделать. Переключаясь в момент L_1 , мы получаем гарантию с точки зрения теории, что все наши вычисления верные. Переключаясь в момент L_2 , мы избегаем любых известных паталогических случаев. Но для большинства операций, когда числа выглядят “случайными”, не являются степенями двойки, не содержат коротких повторяющихся битовых последовательностей, не являются простой функцией от основания, вполне допустимо использование первого преобразования вплоть до момента L_3 .

Но что делать, если мы уверены, что наши числа не “случайны”? Например, в начале или конце теста простоты Ферма. Любое умножение можно сделать «надёжным» с помощью произвольного случайного большого числа r :

$$a \cdot b = (a + r) \cdot (b - r) + r \cdot (a + r - b) \quad (1.5.1)$$

Да, такая операция требует двух умножений вместо одного. Но если мы используем 50 надёжных операций в начале вычислений, 50 надёжных операций в конце вычислений, а миллион операций между ними остаются обычными, то выгода от использования более короткого преобразования перевешивает усложнение нескольких операций.

Такой подход может показаться рискованным, но для многих алгоритмов, применяемых на практике к большим числам, существуют методы относительно быстрой проверки корректности вычислений. Возведение в степень, являющееся основой многих тестов простоты и методов факторизации, может быть проверено алгоритмом Гербица-Ли. Операции над эллиптическими кривыми Эдвардса проверяются соответствием координат уравнению кривой. Используя эти методы, можно если не исключить ошибки округления, то хотя бы их детектировать в маловероятном случае возникновения паталогического числа в середине вычислений. Применение строгих проверок в любом случае желательно, поскольку они помогают также детектировать аппаратные и программные сбои.

Также, в реализации преобразований в комплексном поле желательно использовать методы, уменьшающие ошибки округления для каждой отдельной операции. Одним из таких методов является «балансировка»: использование представления с нормой $-\frac{1}{2}base \leq a_i < \frac{1}{2}base$. Сбалансировать представление довольно просто, достаточно применить операцию:

$$\text{if } a_i > \frac{1}{2}base \text{ then } a_i \leftarrow a_i - base, a_{i+1} \leftarrow a_{i+1} + 1 \quad (1.5.2)$$

Например, число 17 можно представить как $\{2, -3\}$ по основанию 10.

Балансировка в среднем уменьшает абсолютную величину коэффициентов результата умножения на величину порядка $\sqrt{N}$. Но у этого тоже есть исключения, например ситуация гораздо хуже для возведения в квадрат. Это стоит учитывать при выборе N , либо использовать для возведения в квадрат надёжный метод (1.5.1).

1.6 Конечное поле

Приведённый выше алгоритм быстрого умножения остаётся корректным, даже если в качестве поля для коэффициентов взять кольцо вычетов по модулю простого числа, $\mathbb{F} = \mathbb{Z}_p$. Действительно, деление можно заменить на умножение на обратное, а i на первообразный корень из единицы степени 4. Единственное ограничение — в поле $\mathbb{Z}_p$ должен существовать первообразный корень из единицы степени N .

У такого алгоритма есть названия «ДПФ над конечным полем» и «теоретико-численное преобразование» (number-theoretic transform, NTT), хотя с алгоритмической точки зрения он идентичен варианту с комплексными числами. В языках программирования, поддерживающих полиморфизм, можно создать единую реализацию быстрого умножения, которая сможет работать как с $\mathbb{C}$, так и с $\mathbb{Z}_p$ в зависимости от типа входных данных.

Но, разумеется, для оптимальной производительности необходимо учитывать особенности целочисленной арифметики. Прежде всего, коэффициенты результата всё так же ограничены типом данных. Да, здесь нет проблем с ошибками округления, но есть проблемы с ошибками переполнения. Дело в том, что коэффициенты результата имеют двойкий смысл. На стадии умножения они лежат в поле $\mathbb{Z}_p$, но на стадии нормализации подразумевается, что они лежат в $\mathbb{N}$. Конечно, результат всегда будет корректен в $\mathbb{Z}_p$, но если хотя бы к одному коэффициенту результата будет применена модульная операция, то выражение $a \cdot b = P(base)$ будет уже неверно.

Этот факт заставляет использовать достаточно большие p , что не очень удобно. Современные параллельные процессоры (графические ускорители) оптимизированы для 32-битных вычислений. Либо мы используем 32-битное p , что накладывает серьёзные ограничения на N и $base$, либо используем существенно более медленную 64-битную арифметику. Но у этой дилеммы существует изящное решение. Так как результат всегда корректен в $\mathbb{Z}_p$, мы можем использовать несколько относительно небольших p_i , выполнить алгоритм умножения независимо для каждого из модулей, а затем в стадии нормализации восстановить значения коэффициентов по модулю $\prod p_i$ с помощью Китайской теоремы об остатках.

Например, программа Genefer (<https://github.com/galloty/genefer22>) использует 32-битные простые числа $125 \cdot 2^{25} + 1$, $243 \cdot 2^{24} + 1$, $235 \cdot 2^{24} + 1$. Эти простые числа позволяют выполнять преобразования длины 2^{24} по модулю $x^N - 1$ и длины 2^{23} по модулю $x^N + 1$. Восстановленные коэффициенты результата могут иметь до 95 бит, что позволяет выбирать очень большое основание представления.

1.7 Иерархическая память

Современные вычислительные устройства обладают весьма сложной системой хранения данных. Как правило, конкретная топология этой системы скрыта от исполняемых программ, но, тем не менее, она существенно влияет на скорость вычислений. Типичный современный многоядерный центральный процессор имеет на каждом ядре небольшой кеш первого уровня для данных и команд, а также существенно больший кеш второго уровня для данных. Кроме того, в процессоре есть кеш третьего уровня, измеряемый мегабайтами и десятками мегабайт, доступ к которому имеют все ядра в равной степени. По сути, этот кеш является локальной памятью процессора. В неё копируются данные из медленной оперативной памяти, находящейся за пределами процессора. В многопроцессорных системах оперативная память может также подразделяться на «местную» (физически подключенную к данному процессору) и «чужую» (подключенную к другому процессору той же системы). Вся эта сложность спрятана за плоским адресным пространством, выделяемым операционной системой исполняемой программе.

Однако, высокопроизводительные вычисления обязаны учитывать иерархическую организацию памяти на всех уровнях. Нужно следить, чтобы все данные хранились только в местной оперативной памяти, и стараться, чтобы объём данных, необходимых для выполнения одной операции, не превышал кеш третьего уровня. С точки зрения умножения больших чисел это означает, что всё преобразование целиком должно укладываться в кеш третьего уровня, поскольку на каждом уровне рекурсивного алгоритма используются все значения. При использовании многоядерных процессоров возможна ситуация, когда каждое ядро работает над своим преобразованием, которое само по себе уместится в общий кеш, но все вместе они не помещаются. Решением является распараллеливание работы над одним преобразованием на несколько ядер. Возможная неэффективность, вызванная необходимостью синхронизации, более чем компенсируется отсутствием необходимости обращаться к медленной оперативной памяти.

Но и на более низком уровне возможна оптимизация. Для каждого отдельного ядра предпочтительнее иметь все необходимые данные в его собственном кеше второго уровня. По отношению к нему общий кеш третьего уровня является «медленной» памятью. Однако, размер кеша второго уровня весьма ограничен, и преобразования существенной длины в него не помещаются. Именно для такой ситуации был разработан метод, называемый «преобразование в несколько проходов» (multi-pass transform) [1].

Этот метод тоже основан на удобных свойствах корней многочлена $x^N - 1$. В результате применения преобразования несколько раз к одним и тем же данным получаются значения всё того же многочлена $A(x)$ в некоторых корнях многочлена $x^N - 1$. Правильно организовав процесс и домножив данные на недостающие корни, мы можем получить правильный результат используя более короткое преобразование несколько раз.

Для начала, длина преобразования N разбивается на два примерно равных значения $N = N_1 \cdot N_2$. После этого, входные данные рассматриваются как матрица размером $N_1 \times N_2$ и выполняются следу-

ющие шаги:

1. N_1 раз выполняется преобразование длины N_2 .
2. Данные транспонируются в матрицу размером $N_2 \times N_1$.
3. Данные домножаются на соответствующие корни, которые находятся по формуле $i^{jk/N}$.
4. N_2 раз выполняется преобразование длины N_1 .

На каждом шаге мы читаем фиксированное количество данных из медленной памяти в локальную, проводим нужные операции и записываем обратно. Легко увидеть некоторые оптимизации, которые позволяют уменьшить количество обращений к медленной памяти. Третий шаг можно выполнять одновременно с четвёртым. Первые два шага можно объединить в следующую последовательность:

- Копируется столбец высотой N_2 в локальную строку данных.
- Выполняется преобразование длины N_2 .
- Копируется локальная строка данных в столбец высотой N_2 .

В результате этих оптимизаций количество чтений из медленной памяти равно 2, количество записей в медленную память равно 2.

Обратное преобразование использует симметричную последовательность действий. Но для обратного преобразования существует дополнительная оптимизация, существенно улучшающая производительность. Дело в том, что результат умножения больших чисел часто используется только как входные данные для следующей операции умножения. То есть, после нормализации следует ещё одно преобразование. Организовав данные соответствующим образом, эти шаги можно совместить.

- Последний шаг обратного преобразования заполняет локальную строку данных результатом.
- Выполняется нормализация.
- Выполняется первый шаг прямого преобразования.
- Локальная строка данных записывается в медленную память.

Результат оказывается частично преобразованным, что позволяет сэкономить 1 чтение и 1 запись в медленную память. Такая оптимизация даёт до 20% роста производительности на длинных цепочках умножений или возведений в квадрат, что часто происходит в тестах простоты. Всё за счёт учёта особенностей организации иерархической памяти.

Подобные оптимизации возможны и на уровне операций над большими числами. Допустим, необходимо вычислить две формулы $(a + b) \cdot c$ и $(a + b) \cdot d$. Очевидно, что для минимизации количества операций необходимо сначала вычислить и сохранить промежуточную сумму $a + b$. Но такой подход приводит к 6 чтениям и 3 записям. Вычисление формул независимо приводит к 6 чтениям и 2 записям. Для достаточно больших чисел доступ к памяти становится узким местом, и, уменьшив количество обращений к памяти, можно получить лучшую производительность, даже несмотря на большее количество выполненных операций.

1.8 Умножение по модулю

В задачах вычислительной теории чисел, как правило, требуется находить не произведение двух больших чисел, а остаток этого произведения по заданному модулю. Требуется выполнить дополнительную операцию нахождения остатка, то есть в общем случае деление с остатком. К счастью, так как

модуль задан заранее и не меняется, можно заменить дорогостоящую операцию деления на два умножения. Для этого первым шагом мы находим многочлен $D(x)$, называемый псевдообратным к модулю $M(x)$:

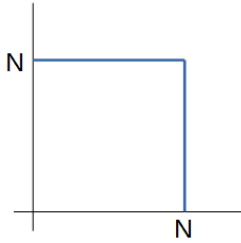
$$D(x) \times M(x) = x^N \quad (1.8.1)$$

Это можно сделать достаточно быстро с помощью метода Ньютона [2]. Заметим, что N должно быть как минимум в два раза больше, чем степень $M(x)$.

Следующим шагом мы вычисляем $T(x) = A(x) \times B(x)$, результат умножения. Если каждый множитель меньше модуля, то и $\deg(T) \leq 2\deg(M) < N$. Теперь нам нужно найти частное $Q(x)$ и остаток $R(x)$, такие что $T(x) = Q(x) \times M(x) + R(x)$. Используя (1.8.1)

$$Q(x) = \frac{T(x) \times D(x)}{x^N}, \quad R(x) = T(x) - Q(x) \times M(x) \quad (1.8.2)$$

Операция деления на x^N реализуется простым отбрасыванием всех коэффициентов с индексом меньше N и уменьшением индексов оставшихся коэффициентов на N .



Рассмотрим алгоритм быстрого умножения по модулю $x^N - 1$. В случае, если сумма степеней множителей превышает N , мы получаем результат, к которому уже применён модуль:

$$P^{mod}(x) = P(x) \bmod (x^N - 1) \quad (1.8.3)$$

Коэффициенты многочленов $P(x)$ and $P^{mod}(x)$ связаны соотношением, которое легко получить, выполнив деление столбиком:

$$\text{mod_}p_k = p_k + p_{N+k} \quad (1.8.4)$$

По сути, та часть, что превышает N , циклически заворачивается и прибавляется к младшим разрядам. Аналогично выглядит соотношение для результата умножения по модулю $x^N + 1$, только сложение в (1.8.4) заменяется на вычитание, благодаря чему такая операция может называться словом «негациклическая».

Для умножения $T(x) \times D(x)$ в (1.8.2) необходимо преобразование длиной как минимум $N + \deg(M)$. Но заметим, что бóльшая часть результата умножения не используется, мы просто отбрасываем N младших коэффициентов, а значит нет никакой необходимости их вычислять. Если занулить первые $\deg(M)$ коэффициентов $T(x)$, то есть установить $p_k = 0$ для младших разрядов, то нужная нам старшая часть при быстром умножении по модулю $x^N - 1$ окажется прибавленной к этим нулевым значениям:

$$Q(x) = (T(x) - T(x) \bmod x^{\deg(M)}) \times D(x) \bmod (x^N - 1) \bmod x^{\deg(M)} \quad (1.8.5)$$

Благодаря этой хитрости, оба умножения в (1.8.2) можно выполнять с помощью того же самого преобразования длины N , что используется и для умножения исходных чисел.

Таким образом, умножение двух больших чисел по модулю оказывается в три раза медленнее, чем просто умножение двух больших чисел. Это весьма печальный факт. Но часто в задачах вычислительной теории чисел в качестве модулей используются не просто случайные большие числа, а числа имеющие определённую алгебраическую форму. Для некоторых из таких форм существуют более эффективные методы нахождения остатка.

Ещё раз вспомним, что быстрое умножение находит результат по модулю $x^N - 1$ или $x^N + 1$. Подставив вместо x основание представления, мы получаем:

$$a \cdot b \bmod (base^N \pm 1) = [P(x) \bmod (x^N \pm 1)](base) \quad (1.8.6)$$

Иными словами, для модулей вида $b^n \pm 1$ мы можем выбрать $base = b$, $N = n$ и больше не делать ничего. Нужный остаток будет получаться сам собой как результат быстрого умножения.

Такой формой обладают числа Ферма $2^{2^n} + 1$ и обобщённые числа Ферма $b^{2^n} + 1$. Для них N оказывается степенью двойки, что позволяет использовать максимально эффективные преобразования по модулю $x^N + 1$. Некоторым минусом данного подхода является то, что N оказывается фиксированным, а значит ограничения на $base$, связанные с используемым типом данных, становятся и ограничениями на b . Для конечных полей эта проблема решается с помощью Китайской теоремы об остатках, но для комплексного поля необходимы более сложные методы. Одним из них является выбор $base = \sqrt{b}$, $N = 2n$, хотя иррациональное основание на практике требует более тщательной работы с точностью представлений.

Обобщённые числа Ферма (generalized Fermat numbers, GFN) являются крайне популярными среди тех, кто ищет большие простые числа. Они в большом количестве присутствуют в списке самых больших известных простых чисел. Прежде всего благодаря скорости модульной операции.

1.9 Взвешенное преобразование

Другими крайне популярными числами являются числа Мерсенна. Большую часть времени с начала эпохи Возрождения самым большим известным простым числом являлось какое-либо из чисел Мерсенна. Они имеют форму $2^n - 1$, но простыми могут быть лишь $2^p - 1$, где p — простое число. Использование простого числа в качестве длины преобразования крайне нежелательно, поскольку это является худшим случаем с точки зрения производительности. В 1994-м году Крендалл и Фагин предложили метод умножения чисел Мерсенна, который они назвали «взвешенное дискретное преобразование с иррациональным основанием» (irrational base discrete weighted transform, IBDWT) [3]. В качестве основания они предлагают использовать $base = 2^{n/N}$, для некоего удобного для нас $N < n$, например степени двойки.

Конечно, первым делом возникает вопрос, каким образом можно построить подобное представление. Наиболее очевидный способ — построить представление как цифры в $2^{n/N}$ -ричной системе счисления, хотя мы и столкнёмся с большими трудностями в поддержании необходимой точности. Крендалл и Фагин предложили метод «взвешивания».

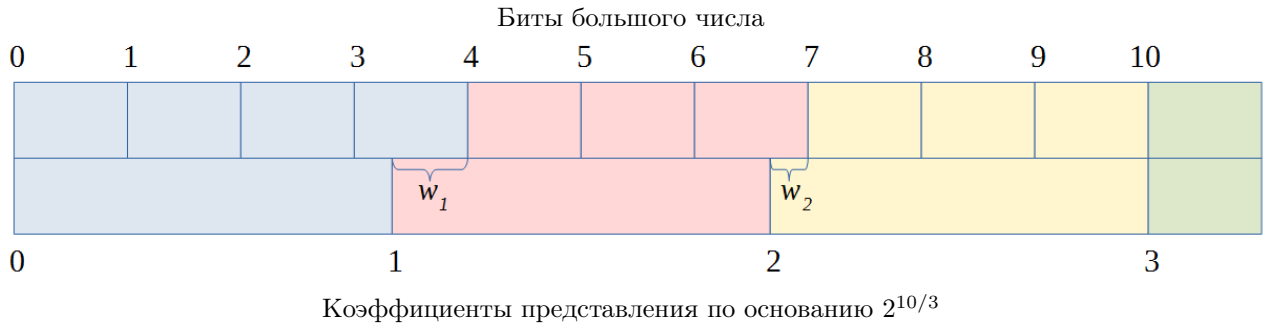


Рис. 1.6: Большое число в двоичном виде и его представление по иррациональному основанию.

Предположим, что мы выбрали основание $base = 2^{10/3}$ (рис. 1.6). Тогда несколько первых членов будут иметь вид

$$A(base) = a_0 + a_1 \cdot 2^{10/3} + a_2 \cdot 2^{20/3} + \dots$$

в то время как двоичное разложение большого числа имеет вид

$$\begin{aligned} a &= a_{[0123]} + a_{[456]} \cdot 2^4 + a_{[789]} \cdot 2^7 + \dots \\ &= a_{[0123]} + a_{[456]} \cdot 2^{4-10/3} \cdot 2^{10/3} + a_{[789]} \cdot 2^{7-20/3} \cdot 2^{20/3} + \dots \end{aligned}$$

Таким образом, мы можем построить представление используя битовые подстроки большого числа и иррациональные степени двойки:

$$a_0 = \mathbf{a}_{[0123]}, \quad a_1 = \mathbf{a}_{[456]} \cdot 2^{4-10/3}, \quad a_2 = \mathbf{a}_{[789]} \cdot 2^{7-20/3}, \dots$$

Да, коэффициенты представления будут больше основания. Но в этом нет никакой проблемы, главное, что $A(\text{base}) = \mathbf{a}$. К тому же, превышение будет всегда меньше чем в 2 раза (меньше чем на 1 бит). Иррациональные степени двойки, на которые домножаются битовые подстроки, называются «весами» и в общем случае вычисляются как

$$w_i = 2^{\lceil in/N \rceil - in/N} \in [1, 2) \quad (1.9.1)$$

В результате, мы получаем представление

$$a_i = w_i \mathbf{a}_{[\lceil in/N \rceil, \lceil (i+1)n/N \rceil)} \quad (1.9.2)$$

Легко увидеть, что длина битовых подстрок равна либо $\lfloor n/N \rfloor$, либо $\lceil n/N \rceil$ (3 и 4 в приведённом выше примере). И они так же могут быть отбалансированы с помощью (1.5.2).

Взвешенное представление обладает свойством, что $\frac{a_i}{w_i}$ является целым числом. Но что можно сказать о произведении двух таких многочленов? Как мы помним из умножения столбиком, каждое p_k является суммой произведений вида $a_i b_j$, где $i + j = k$. Тогда $\frac{p_k}{w_k}$ будет суммой целых чисел (произведений битовых подстрок), умноженных на следующие веса:

$$\begin{aligned} \frac{w_i w_j}{w_{i+j}} &= 2^{\lceil in/N \rceil - in/N + \lceil jn/N \rceil - jn/N - \lceil (i+j)n/N \rceil + (i+j)n/N} \\ &= 2^{\lceil in/N \rceil + \lceil jn/N \rceil - \lceil in/N + jn/N \rceil} \end{aligned} \quad (1.9.3)$$

Сумма двух округлённых вверх величин либо равна, либо на единицу больше округлённой вверх суммы величин. Таким образом, (1.9.3) равно либо 1, либо 2. $\frac{p_k}{w_k}$ всегда является целым числом. Это верно даже при умножении по модулю $x^N \pm 1$. Это следует из (1.8.4) и того факта, что $w_i = w_{N+i}$.

Используя (1.2.2) и (1.9.1), мы можем записать результат умножения следующим образом:

$$\mathbf{a} \cdot \mathbf{b} = P(2^{n/N}) = \sum p_k 2^{kn/N} = \sum \frac{p_k}{w_k} 2^{\lceil kn/N \rceil} \quad (1.9.4)$$

p_k и $2^{kn/N}$ являются иррациональными числами, но $\frac{p_k}{w_k}$ и $2^{\lceil kn/N \rceil}$ являются целыми числами, поскольку вся иррациональность взаимоуничтожается. Нам остаётся лишь применить операцию округления (и посчитать ошибку округления для контроля правильности выбора N). Это означает, что в стадии нормализации мы можем работать только с целыми числами.

Напомним, что используя описанное выше взвешенное преобразование, мы автоматически получаем остаток от деления на $\text{base}^N \pm 1 = 2^n \pm 1$. Но подобное же преобразование можно также построить для чисел Прота $k \cdot 2^n + 1$ и чисел $k \cdot 2^n - 1$, которые иногда называют числами Ризеля. Для таких чисел выбирается основание $\text{base} = 2^{(n+\log_2 k)/N}$. Веса вычисляются как

$$w_i = 2^{\lceil i(n+\log_2 k)/N \rceil - i(n+\log_2 k)/N} \in [1, 2) \quad (1.9.5)$$

К сожалению, $w_i \neq w_{N+i}$:

$$\begin{aligned} \frac{w_{N+i}}{w_i} &= 2^{\lceil n+\log_2 k + i(n+\log_2 k)/N \rceil - n - \log_2 k - i(n+\log_2 k)/N - \lceil i(n+\log_2 k)/N \rceil + i(n+\log_2 k)/N} \\ &= \frac{1}{k} 2^{\lceil \log_2 k + i(n+\log_2 k)/N \rceil - \lceil i(n+\log_2 k)/N \rceil} \end{aligned} \quad (1.9.6)$$

Это означает, что у $\frac{p_t}{w_t}$ будет множитель $\frac{1}{k}$, и эта величина уже не будет целой. Это существенно усложняет стадию нормализации. Существует способ справиться с этим, заключающийся в том, что

все большие числа предварительно умножаются на k^{-1} . Для того, чтобы результат умножения больших чисел обладал таким же множителем, его необходимо дополнительно умножить на k в стадии нормализации.

$$(a \cdot b)k^{-1} = k(ak^{-1} \cdot bk^{-1}) = k \sum p_t 2^{t(n+\log_2 k)/N} = \sum \frac{kp_t}{w_t} 2^{\lceil t(n+\log_2 k)/N \rceil} \quad (1.9.7)$$

При этом $\frac{kp_t}{w_t}$ является целым числом, и стадия нормализации может оставаться целочисленной.

Взвешенное преобразование также может быть использовано и для чисел вида $k \cdot b^n \pm 1$. Но основание $base = 2^{(n \log_2 b + \log_2 k)/N}$ не подходит, поскольку $\frac{w_{N+i}}{w_i}$ оказывается иррациональным, поэтому используется основание $base = b^{(n+\log_b k)/N}$. Это означает, что для построения представления используются не битовые подстроки, а подстроки из представления с основанием b . При этом веса вычисляются аналогично (1.9.5), только основание степени заменяется на b .

$$w_i = b^{\lceil i(n+\log_b k)/N \rceil - i(n+\log_b k)/N} \in [1, b) \quad (1.9.8)$$

Для чисел вида $b^n \pm c^m$ используется модуль $b^n c^{-m} \pm 1$ [4]. При этом веса учитывают оба основания:

$$w_i = b^{\lceil in/N \rceil - in/N} c^{\lceil -im/N \rceil + im/N} \in [1, bc) \quad (1.9.9)$$

Получившиеся веса лежат в более широком диапазоне, зависящем в том числе от c . Но зато $w_i = w_{N+i}$ и $\frac{w_i w_j}{w_{i+j-N}} = c$ в простейшем случае $m = 1$ (что равнозначно $\text{mod_}p_k = p_k \mp cp_{n+k}$, что равнозначно нахождению остатка от деления столбиком на $b^n \pm c$). А значит, в стадии нормализации мы работаем только с целыми числами, и нет необходимости совершать какие-то дополнительные действия для поддержки $c \neq 1$.

Для реализации взвешенного преобразования проще всего использовать вычисления в комплексном поле. Однако, при взгляде на формулы, в которых даже показатели степеней являются иррациональными, может возникнуть вопрос, какая точность необходима для получения правильного результата? Действительно, при использовании типа данных с двойной точностью веса вычисляются с довольно серьёзной погрешностью, которая накапливается в ходе дальнейших вычислений. Но у нас нет необходимости выполнять сложную работу по оценке погрешности, поскольку мы в точности знаем величину ошибки. Это дробная часть числа, которое должно быть целым. Именно возможность отбросить накопленную ошибку простой операцией округления и обеспечивает успех взвешенного преобразования на практике.

Использование поля $\mathbb{Z}_p$ также возможно, но на практике требует довольно больших простых модулей и уже не может быть реализовано в 32-битной арифметике. Даже 64-битная арифметика позволяет реализовывать лишь самые простые виды взвешенного преобразования, вроде $k = 1$.

Библиотека GWNuM (<https://www.mersenne.org/download/>) реализует взвешенное преобразование в комплексном поле для чисел вида $k \cdot b^n + c$ для широкого диапазона параметров.

1.10 Свободное пространство слов

В качестве длины преобразования N удобнее всего выбирать степень двойки, на практике также используются длины с небольшими простыми делителями 3 и 5. Это делается для того, чтобы при смене преобразования оно не увеличивалось сразу в два раза, а росло более плавно. Тем не менее, плавности всё равно не хватает — после смены преобразования уровень ошибок округления резко падает, становится гораздо ниже пороговой величины. Разница между текущими средними ошибками округления и пороговой величиной называется «свободное пространство слов». На рис. 1.4 это область выше закрашенной и ниже S . Эту разницу можно понимать как некое неиспользуемое пространство, остающееся в каждом коэффициенте (слове) результата до нормализации. И мы можем это пространство использовать.

Вспомним, что при восстановлении коэффициентов результата с помощью обратного преобразования, мы нигде не использовали тот факт, что это произведение двух многочленов (1.2.1). Для нас важны только такие параметры, как степень многочлена и размер коэффициентов. Есть несколько других операций, помимо $\mathbf{a} \cdot \mathbf{b}$, которые дают примерно такую же степень и размер коэффициентов результата. Например, $\mathbf{a} \cdot \mathbf{b} + \mathbf{c}$ или $(\mathbf{a} + \mathbf{b}) \cdot \mathbf{c}$ или даже $\mathbf{a} + \mathbf{b}$. Иными словами, мы можем вычислить целую формулу (не более, чем второй степени) и получить результат, использовав обратное преобразование только один раз.

Определим следующие функции, преобразующие числа:

transform Выполняет прямое преобразование $\overline{A} = \text{transform}(A)$.

inv_transform Выполняет обратное преобразование $A = \text{inv_transform}(\overline{A})$.

carry Выполняет нормализацию и модульную операцию (если необходимо) $A = \text{carry}(A)$.

Аргументами этих функций являются векторы, содержащие либо многочлен $A(x)$, либо его значения в соответствующих корнях: $\overline{A} = \{A(r_i)\}$. Все арифметические операции над векторами выполняются поэлементно.

Мы можем записать произведение двух больших чисел $\mathbf{a} \cdot \mathbf{b}$ в следующем виде:

$$\text{carry}(\text{inv_transform}(\text{transform}(A) \cdot \text{transform}(B))) \quad (1.10.1)$$

Выражение $\mathbf{a} \cdot \mathbf{b} + \mathbf{c} \cdot \mathbf{d}$ можно вычислить следующими способами, используя уже преобразованные данные:

$$\text{carry}(\text{carry}(\text{inv_transform}(\overline{A} \cdot \overline{B})) + \text{carry}(\text{inv_transform}(\overline{C} \cdot \overline{D}))) \quad (1.10.2)$$

$$\text{carry}(\text{inv_transform}(\overline{A} \cdot \overline{B} + \overline{C} \cdot \overline{D})) \quad (1.10.3)$$

Во втором случае выполняется одно обратное преобразование вместо двух и одна нормализация вместо трёх.

Важно понимать, что каждая арифметическая операция над преобразованными данными увеличивает размер коэффициентов (слов), поскольку нормализация невозможна до выполнения обратного преобразования.

$$\overline{A} + \overline{B} \neq \text{transform}(\text{carry}(\text{inv_transform}(\overline{A} + \overline{B}))) \quad (1.10.4)$$

Это значит, что для дополнительных слагаемых необходимо наличие свободного пространства в словах. Насколько много — трудно оценить теоретически, но легко посчитать практически, вычисляя среднюю ошибку округления для различных формул. Библиотека GWNuM предлагает пользователям функции, способные ответить на вопрос, безопасно ли вычислять ту или иную формулу с помощью данного N и данного размера больших чисел.

В ходе наработки статистики по ошибкам округления для различных формул, были обнаружены некоторые закономерности. Например, $\mathbf{a}^2$ и $(\mathbf{a} + \mathbf{b}) \cdot \mathbf{c}$ требуют примерно одинаковое свободное пространство. Это означает, что если преобразование можно использовать для вычисления квадрата большого числа, то его можно использовать и для вычисления $(\mathbf{a} + \mathbf{b}) \cdot \mathbf{c}$. Некоторые подобные классы формул указаны в следующей таблице, приведённые по возрастанию необходимого свободного пространства:

$\mathbf{a}^2, \quad (\mathbf{a} + \mathbf{b}) \cdot \mathbf{c}, \quad \mathbf{a} \cdot \mathbf{b} + \mathbf{c}, \quad \mathbf{a} \cdot \mathbf{b} + \mathbf{c} \cdot \mathbf{d}$
$(\mathbf{a} + \mathbf{b} + \mathbf{c}) \cdot \mathbf{d}, \quad \mathbf{a}^2 + \mathbf{b} \cdot \mathbf{c}, \quad (\mathbf{a} + \mathbf{b}) \cdot \mathbf{c} + \mathbf{d} \cdot \mathbf{e}$
$\mathbf{a}^2 + \mathbf{b}^2, \quad (\mathbf{a} + \mathbf{b}) \cdot (\mathbf{c} + \mathbf{d}), \quad (\mathbf{a} + \mathbf{b} + \mathbf{c} + \mathbf{d}) \cdot \mathbf{e}, \quad (\mathbf{a} + \mathbf{b}) \cdot \mathbf{c} + (\mathbf{d} + \mathbf{e}) \cdot \mathbf{f}$
$\mathbf{a}^2 + \mathbf{a}^2, \quad (\mathbf{a} + \mathbf{b})^2, \quad (\mathbf{a} + \mathbf{b} + \mathbf{c} + \mathbf{d} + \mathbf{e} + \mathbf{f} + \mathbf{g} + \mathbf{h}) \cdot \mathbf{i}, \quad (\mathbf{a} + \mathbf{b} + \mathbf{c} + \mathbf{d}) \cdot (\mathbf{e} + \mathbf{f}),$ $(\mathbf{a} + \mathbf{b}) \cdot (\mathbf{c} + \mathbf{d}) + (\mathbf{e} + \mathbf{f}) \cdot (\mathbf{g} + \mathbf{h}), \quad (\mathbf{a} + \mathbf{b} + \mathbf{c} + \mathbf{d}) \cdot \mathbf{e} + (\mathbf{f} + \mathbf{g} + \mathbf{h} + \mathbf{i}) \cdot \mathbf{j}$

Возможность или невозможность использовать эти формулы для конкретного большого числа приводит к тому, что некоторые алгоритмы могут иметь несколько реализаций на выбор, в зависимости от доступного свободного пространства слов. Самая быстрая реализация требует больше всего свободного пространства. Если его нет, то можно либо увеличить N , что существенно замедлит вычисления, либо использовать другую реализацию, лишь ненамного медленнее. Например, алгоритм удвоения точки на эллиптической кривой Эдвардса может иметь четыре реализации [5].

Есть ещё одна интригующая возможность. Если увеличить N в два раза, то можно безопасно вычислять формулы четвёртой степени. В том же алгоритме удвоения точки на эллиптической кривой Эдвардса степень всех формул равна четырём. К сожалению, там используется много квадратов и четвёртых степеней, что крайне негативно влияет на ошибки округления, преобразования длины $2N$ оказывается в большинстве случаев недостаточно. Но когда его хватает, получившийся алгоритм работает на 5% быстрее классического.

Алгоритмы и операции, выполняющиеся над преобразованными большими числами, мы называем «вложенными», поскольку они выполняются «внутри» преобразования. Особенностью таких алгоритмов является то, что они не зависят от вида преобразования или наличия модульной операции. Если она есть (либо как отдельная операция, либо как взвешенное преобразование), то после обратного преобразования мы сразу же получаем ответ по нужному нам модулю, без необходимости применять модуль к каждой составляющей вычисляемой формулы.

1.11 Вложенное умножение

Среди всех вложенных алгоритмов особый интерес вызывает вложенное умножение многочленов. На самом деле, умножение многочленов широко используется в различных методах разложения чисел на простые множители, в том числе больших чисел. Как правило, в таких задачах необходимо находить коэффициенты результата по модулю большого числа. Допустим, нам даны два многочлена $A(y)$ и $B(y)$ с большими коэффициентами a_i и b_j , а также большой модуль m . Тогда мы можем вычислить произведение $P(y)$ степени $M - 1$ с помощью умножения столбиком:

$$P(y) = A(y) \times B(y) \quad (1.11.1)$$

$$p_k = \sum_{i+j=k} a_i \cdot b_j \bmod m \quad (1.11.2)$$

Чтобы найти коэффициенты результата, можно выполнять алгоритм быстрого умножения $a_i \cdot b_j \bmod m$ для каждой пары коэффициентов отдельно, а потом сложить результаты. Но (1.11.2) является формулой второй степени, а значит мы можем вычислить её, используя только одно обратное преобразование. Конечно, нам понадобится дополнительное свободное пространство в словах, но не так уж и много. Напомним, что при увеличении M , в коэффициентах произведения увеличивается лишь число слагаемых (рис. 1.1), размер слагаемых остаётся тем же самым. А значит, размер необходимого свободного пространства зависит от $\log_2 M$. На практике, даже для очень больших степеней многочленов достаточно использовать преобразование на ступеньку больше минимального. Получившийся алгоритм мы называем «вложенное умножение столбиком».

Рассмотрим подробнее, как этот алгоритм работает. На первом этапе выполняется преобразование всех больших чисел. Обозначим получившиеся большие коэффициенты как $\bar{a}_i$ и $\bar{b}_j$. Каждый из них состоит из N элементов, которые можно обозначить как $\bar{a}_{i\ell}$ и $\bar{b}_{j\ell}$. Далее для каждого ℓ вычисляется $\bar{p}_{k\ell}$ с помощью следующей формулы:

$$\bar{p}_{k\ell} = \sum_{i+j=k} \bar{a}_{i\ell} \cdot \bar{b}_{j\ell} \quad (1.11.3)$$

Здесь ℓ является фиксированной величиной. А значит, мы можем переписать эту формулу в терминах многочленов:

$$P_\ell(y) = A_\ell(y) \times B_\ell(y) \quad (1.11.4)$$

Заметим, что в этот момент мы перестаём считать данные преобразованными. Теперь для нас это просто два многочлена, которые нужно перемножить. Степень результата умножения $P_\ell(y)$ будет всё той же $M - 1$. Для вычисления этого многочлена можно использовать описанное выше быстрое умножение по модулю $y^M - 1$, только без стадии нормализации. Для каждого ℓ с помощью преобразования длины M вычисляются $\bar{A}_\ell$ и $\bar{B}_\ell$, затем вычисляется $\bar{P}_\ell$, к которому применяется обратное преобразование длины M . В конце, обратное преобразование длины N применяется к каждому из $\bar{p}_k$. Получившийся алгоритм мы называем «вложенное быстрое умножение» или «вложенное БПФ» или даже «БПФ внутри БПФ».

Важным отличием вложенного быстрого умножения от многопроходного быстрого умножения является тот факт, что вложенное преобразование и внешнее преобразование независимы друг от друга. Внешнее преобразование может быть взвешенным преобразованием по модулю $x^N + 1$, а вложенное преобразование по модулю $y^M - 1$. Совершенно разные множества точек $\{r_i\}$. Более того, при реализации вложенного умножения, выбор вложенного преобразования зависит только от M и совершенно не зависит от внешнего преобразования. Достаточно знать, что вложенное преобразование нужно применить N раз.

Как и для любого вложенного алгоритма, внешняя модульная операция применяется только один раз для каждого коэффициента результата, даже если он состоит из миллиона слагаемых. Таким образом, при малых M и больших N время выполнения алгоритма определяется скоростью внешнего преобразования (и модульной операции, если она есть в явном виде), сложность алгоритма близка к $O(M)$. Это особенно ярко видно, когда преобразование длины M помещается в кеш второго уровня, а преобразование длины N требует многопроходного алгоритма и обращения к медленной памяти. Иными словами, в определённых условиях умножение многочленов выполняется за линейное время (рис. 1.7).

С другой стороны, при больших M проявляется другое полезное свойство вложенного умножения. В памяти нужно хранить только одно преобразование длины M , данные для каждого следующего ℓ (или “линия”) могут записываться в один и тот же участок памяти. Если степень многочленов $A(y)$ и $B(y)$ меньше M , то нет необходимости хранить ведущие нули как коэффициенты, они могут быть добавлены непосредственно в каждую индивидуальную линию перед преобразованием. Более того, коэффициенты многочленов $A(y)$ и $B(y)$ можно использовать для хранения многочлена $P(y)$, поскольку после копирования линии содержимое оригинальных участков памяти больше никак не используется. Всё это означает, что даже для очень больших степеней многочленов, количество памяти, необходимое для их перемножения, относительно невелико. На практике это делает возможным перемножение двух многочленов степени миллион, каждый коэффициент которых содержит больше тысячи цифр, на персональном компьютере всего за несколько секунд.

Использование вложенного преобразования по модулю $y^M - 1$ равнозначно применению модуля $y^M - 1$ к многочлену $P(y)$. Это позволяет выполнять операции по модулю большого многочлена точно так же, как описано в разделе 1.8. Кроме того, в ситуации, когда нужно получить только часть коэффициентов результата, можно сэкономить на обратных преобразованиях и просто не выполнять их для ненужных коэффициентов.

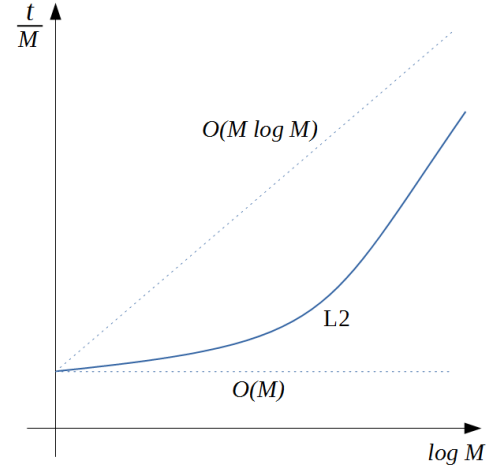


Рис. 1.7: Производительность вложенного умножения многочленов степени M до и после превышения размера кеша второго уровня L2.

Ещё одной возможной оптимизацией является сохранение преобразованных данных для каждой линии определённого многочлена в целях повторного использования. Например, если многочлен $B(y)$ используется без изменения в нескольких операциях умножения, то можно вместо него хранить в памяти $\{\bar{B}_\ell\}$. Однако такой подход требует больше памяти, если $\deg(B) < M$.

Вложенное умножение реализовано в пакете `polymult` библиотеки `GWNum` (<https://www.mersenne.org/download/>) для всех поддерживаемых библиотекой больших чисел.

1.12 Реализация

Как уже упоминалось, программная реализация описанных алгоритмов очень сильно зависит от конкретного вычислительного устройства. Для достижения максимальной производительности нужно учитывать иерархическую организацию памяти и наличие векторных инструкций процессора. Помимо этого, немаловажным фактором является выбор языка программирования.

Высокоуровневые языки программирования редко предоставляют возможность работы с векторными инструкциями процессоров, а если такая возможность есть, то результат может сильно зависеть от версии и года выпуска компилятора. Известны случаи, когда использование более поздней версии компилятора приводило к падению производительности. Дело в том, что одну и ту же последовательность операций можно реализовать разными инструкциями. Например, операнд из оперативной памяти может использоваться напрямую, а может сначала быть загружен в регистр процессора. И какой из этих вариантов быстрее может зависеть от архитектуры конкретного процессора или семейства процессоров. Более того, стратегия оптимизации, которой придерживается компилятор, может зависеть в том числе от наиболее популярных классов задач, решаемых пользователями на момент выпуска компилятора. Всё это приводит к тому, что производительность программы, написанной на высокоуровневом языке, может меняться от года к году.

Использование ассемблера, с другой стороны, сопряжено со множеством своих проблем. Прежде всего, это необходимость написания отдельного кода для каждого варианта инструкций процессоров, для каждой архитектуры процессоров, для каждого вида преобразований. Пропадает любая возможность портирования программы на другие вычислительные устройства. Исходный код становится крайне запутанным и требует специалистов высочайшего класса для поддержки. Но в результате достигается наивысшая производительность вычислений, доступная на используемом аппаратном обеспечении.

Компромиссом является написание модулей преобразований и нормализаций на ассемблере, с предоставлением доступа к этим модулям из языков более высокого уровня. В свою очередь, следующий уровень абстракции прячет все аппаратно-зависимые вызовы за единым программным интерфейсом элементарных операций.

С точки зрения доступа к памяти, наиболее эффективно преобразование выполняется по месту хранения числа, перезаписывая данные. Для преобразований над конечным полем размер исходного числа и преобразованных данных полностью совпадает (с учётом ведущих нулей там, где это необходимо). При использовании же комплексного поля исходные коэффициенты вещественны, а преобразованные данные комплексны. Но рассматриваемые нами преобразования обладают эрмитовой симметрией, что означает, что симметричные комплексные значения отличаются только знаком мнимой части. Это позволяет в два раза уменьшить необходимый объём памяти. В результате, преобразованные данные по модулю $x^N - 1$ состоят из двух вещественных значений и $N/2 - 1$ пар комплексных чисел, преобразованные данные по модулю $x^N + 1$ состоят из $N/2$ пар комплексных чисел. И те, и другие данные можно хранить в том же участке памяти, что и N исходных коэффициентов.

Программный интерфейс должен уделять много внимания отслеживанию текущего состояния всех участвующих в операции чисел. Большое число может храниться либо в исходном представлении, либо в преобразованном виде, либо в частично преобразованном виде, как описано в разделе 1.7. Выполнение

элементарной операции может менять состояние не только результата, но и всех остальных операндов. Например, при умножении вида $A = A \times B$, в число A записывается непреобразованный или частично преобразованный результат, а число B становится преобразованным $\bar{B}$. В случае, если пользователь хочет сохранить B непреобразованным, он должен либо сделать копию, либо каким-либо образом сообщить программному интерфейсу о своём желании, чтобы тот использовал временное число для хранения $\bar{B}$. То же самое относится к выполнению частичного преобразования во время нормализации, пользователи должны сообщать нужно ли это делать или нет. Это зависит от того, как число будет использоваться в дальнейшем. Если в операции умножения, то частичное преобразование даст прирост производительности. Но некоторые операции требуют непреобразованное число, например конвертация в двоичную форму или сложение с нормализацией. Если же число уже преобразовано, то потребуется выполнить обратное преобразование (что иногда проще реализовать используя операцию умножения на 1).

Как уже говорилось, для выполнения сложения с нормализацией необходимы непреобразованные числа. Можно выполнить и сложение двух преобразованных чисел, но результат этой операции будет иметь немного другой смысл. Нормализация такого результата невозможна, и такое число безопасно использовать в операции умножения только при наличии достаточного свободного пространства слов, как обсуждалось в разделе 1.10. Программный интерфейс должен отслеживать количество ненормализованных сложений и предоставлять инструменты оценки безопасности дальнейших операций.

Помимо сложения и вычитания, программный интерфейс может предоставлять совмещённую операцию $A' = A + B$, $B' = A - B$. Такая операция легко реализуется поэлементно и не требует сохранения временных копий чисел. Некоторые алгоритмы, например описанные в [5], могут с выгодой использовать подобную операцию, особенно её ненормализованную версию.

В следующей таблице представлены элементарные операции, наличие которых желательно в программном интерфейсе. Функция `carry()` помимо нормализации и модульной операции может опционально выполнять умножение на маленькую константу, добавление маленькой константы и частичное преобразование.

$X \leftarrow A + B$	$\bar{X} \leftarrow \text{transform}(A)$
$X \leftarrow A - B$	$X \leftarrow \text{carry}(\text{inv_transform}(\bar{A}))$
$X, Y \leftarrow A \pm B$	$X \leftarrow \text{carry}(\text{inv_transform}(\bar{A} \cdot \bar{B}))$
$X \leftarrow \text{carry}(A + B)$	$X \leftarrow \text{carry}(\text{inv_transform}((\bar{A} + \bar{B}) \cdot \bar{C}))$
$X \leftarrow \text{carry}(A - B)$	$X \leftarrow \text{carry}(\text{inv_transform}((\bar{A} - \bar{B}) \cdot \bar{C}))$
$X, Y \leftarrow \text{carry}(A \pm B)$	$X \leftarrow \text{carry}(\text{inv_transform}(\bar{A} \cdot \bar{B} + \bar{C}))$
$\bar{X} \leftarrow \bar{A} + \bar{B}$	$X \leftarrow \text{carry}(\text{inv_transform}(\bar{A} \cdot \bar{B} - \bar{C}))$
$\bar{X} \leftarrow \bar{A} - \bar{B}$	$X \leftarrow \text{carry}(\text{inv_transform}(\bar{A} \cdot \bar{B} + \bar{C} \cdot \bar{D}))$
$\bar{X}, \bar{Y} \leftarrow \bar{A} \pm \bar{B}$	$X \leftarrow \text{carry}(\text{inv_transform}(\bar{A} \cdot \bar{B} - \bar{C} \cdot \bar{D}))$

Число X может совпадать с любым из A, B, C, D . Это может влиять на конкретную реализацию операции.

Глава 2

Возведение в степень

Возведение большого числа a в большую степень e по модулю большого числа m является одной из самых популярных задач, выполняемых с использованием больших чисел. Многие тесты простоты используют возведение в степень в том или ином виде. В литературе [6] описано много алгоритмов возведения в степень, мы рассмотрим некоторые из них применительно к большим числам.

2.1 Двоичная степень

Возведение в большую степень e , состоящую из миллионов десятичных цифр, может показаться пугающей задачей. Однако, она относительно легко решается благодаря тому, что все операции выполняются по модулю, а также тому, что основной операцией является возведение в квадрат. Возводя число в квадрат, мы удваиваем показатель степени. Последовательно применяя эту операцию, мы можем достичь показателя степени, сравнимого с e , за $\log_2 e$ шагов.

Число e может быть представлено в двоичном виде как

$$e = e_0 + e_1 2 + e_2 2^2 + e_3 2^3 + \dots \quad (2.1.1)$$

При этом нас не интересуют биты $e_i = 0$, они не оказывают никакого влияния на результат. Сам же результат может быть записан как

$$\begin{aligned} a^e \bmod m &= a^{e_0} \cdot a^{e_1 2} \cdot a^{e_2 2^2} \cdot a^{e_3 2^3} \cdot \dots \bmod m \\ &= \prod_{e_i \neq 0} a^{2^i} \bmod m \end{aligned} \quad (2.1.2)$$

Иными словами, можно последовательно возводить число a в квадрат и если i -ый бит числа e не равен нулю, то домножать временную переменную на текущее значение.

Algorithm 2.1: Возведение в степень справа налево.

Output: $r = a^e \bmod m$

```
1   $r \leftarrow 1, \quad s \leftarrow a$ 
2  for  $i \leftarrow 0$  to  $\lfloor \log_2 e \rfloor$  do
3      if  $e_i \neq 0$  then  $r \leftarrow r \cdot s \bmod m$ 
4       $s \leftarrow s^2 \bmod m$ 
5  end
```

Этот алгоритм называется возведением в степень справа налево. Подразумевается, что самый младший бит двоичной записи числа e находится справа, а самый старший бит слева.

Существует и противоположный алгоритм — возведение в степень слева направо. Как следует из названия, результат собирается начиная со старшего бита e_j и до тех пор, пока j не уменьшится до

нуля. На каждом шаге временная переменная возводится в квадрат, удваивая показатель степени. Таким образом, значение на шаге j к концу работы алгоритма будет возведено в квадрат j раз. Поэтому, если очередной бит числа e не равен нулю, то временная переменная домножается не на полное значение множителя $a^{2^j} \bmod m$, а только на a . То есть, на каждом шаге временная переменная содержит значение $a^{\lfloor e/2^j \rfloor} \bmod m$.

Algorithm 2.2: Возведение в степень слева направо.

Output: $r = a^e \bmod m$

```

1   $r \leftarrow 1$ 
2  for  $j \leftarrow \lfloor \log_2 e \rfloor$  to 0 do
3       $r \leftarrow r^2 \bmod m$ 
4      if  $e_j \neq 0$  then  $r \leftarrow r \cdot a \bmod m$ 
5  end
```

Для примера рассмотрим возведение в степень $314 = 100111010_2$.

Алгоритм 2.1									
i	0	1	2	3	4	5	6	7	8
e_i	0	1	0	1	1	1	0	0	1
s	a	a^2	a^4	a^8	a^{16}	a^{32}	a^{64}	a^{128}	a^{256}
r	1	a^2	a^2	a^{10}	a^{26}	a^{58}	a^{58}	a^{58}	a^{314}

Алгоритм 2.2									
j	8	7	6	5	4	3	2	1	0
e_j	1	0	0	1	1	1	0	1	0
r	a	a^2	a^4	a^9	a^{19}	a^{39}	a^{78}	a^{157}	a^{314}

В каждом из алгоритмов выполняется одинаковое число операций: $\lfloor \log_2 e \rfloor + 1$ возведений в квадрат и некоторое количество умножений, равное числу ненулевых e_i (вес Хемминга). Если показатель степени сам является степенью двойки, то алгоритмы становятся идентичными. Для произвольных показателей вес Хемминга в среднем равен $\frac{1}{2} \log_2 e$.

При решении практических задач часто встречается ситуация, когда e и m являются большими числами, тогда как a является небольшой константой. Например, в тесте простоты Ферма в качестве a часто берут 2 или 3. Тогда в алгоритме 2.2 умножение на a становится умножением на небольшую константу, что может быть реализовано в стадии нормализации практически бесплатно. Напротив, умножение в алгоритме 2.1 всегда является умножением двух больших чисел. В результате, для произвольных степеней производительность алгоритма 2.1 на 50% ниже, чем алгоритма 2.2, даже несмотря на одинаковое с математической точки зрения число операций. Именно на такие эффекты, обусловленные особенностями операций с большими числами, мы и будем обращать внимание в этой главе.

2.2 Проверка Гербица

Помимо производительности, другим немаловажным свойством вычислений является надёжность. Ошибка в одном единственном бите может иметь далеко идущие последствия. Одно из наиболее популярных применений больших чисел — поиск простых чисел из миллионов цифр. Из-за размера, проверка одного числа-кандидата на простоту может выполняться несколько дней. Малейшая ошибка приводит к неверному результату теста, и простое число может быть пропущено. Существуют математические гипотезы, для доказательства которых нужно найти несколько простых чисел, например гипотеза Серпинского. В этих гипотезах заданы последовательности, числа в которых растут очень быстро. Случайный пропуск простого числа сегодня может означать, что следующее простое число

в последовательности будет найдено через сотни лет, и за это время на его поиск будет потрачено огромное количество энергии.

Ситуация усугубляется тем, что как правило поиск таких простых чисел выполняется добровольцами в рамках распределённых проектов. Это означает, что вычислительные устройства находятся не в контролируемых условиях датацентров, а находятся в домах или на рабочих местах энтузиастов. Среди этих вычислительных устройств неизбежно будет некоторое количество неисправных, перегреваемых или плохо защищённых от влияний среды. Для выявления таких устройств традиционно использовалась тактика двойной проверки, когда один и тот же тест запускается на двух несвязанных устройствах, и результаты сравниваются. Но такой подход требует выполнения в два раза большего количества работы, чем необходимо. Поэтому всегда велись поиски более экономной тактики. Например, квалификация устройств, когда двойная проверка выполняется только для первых нескольких тестов. У такого подхода тоже есть недостатки, он не защищает от единичных сбоев, вызванных, например, неожиданным повышением температуры вычислительного устройства из-за жаркой погоды или случайно выключенного кондиционера. Подобный сбой произошёл в 2014 году при тесте числа $281 \cdot 2^{2051865} + 1$. Вычислительное устройство с хорошей репутацией ошибочно признало это число составным. И только в 2022 году при перепроверке старых данных было установлено, что это число простое.

Для решения этой проблемы Гербицем был предложен метод математически строгой проверки возведения в степень с показателем вида 2^n [7]. Такую проверку можно применять для теста Ферма чисел Мерсенна, чисел Прота, чисел “Ризеля”. Метод позволяет выявлять не только аппаратные сбои, но и программные ошибки в реализации операций над большими числами. Накладные расходы, связанные с дополнительными вычислениями, в общем случае не превышают 1%, что выгодно отличает метод от двойной проверки.

Целью проверки Гербица является доказательство корректности вычисления каждого L возведений в квадрат. Допустим, мы посчитали следующие значения:

$$\begin{aligned} a^{2^L} &= (a)^{2^L} \\ a^{2^{2L}} &= (a^{2^L})^{2^L} \end{aligned} \quad (2.2.1)$$

Мы можем проверить корректность вычислений повторив их, то есть выполнив $2L$ возведений в квадрат. Но если мы перемножим эти два уравнения, то должны получить:

$$a^{2^L} \cdot a^{2^{2L}} \stackrel{?}{=} (a \cdot a^{2^L})^{2^L} \quad (2.2.2)$$

Выполнив L возведений в квадрат, мы можем проверить корректность сразу двух значений, сэкономив 50% работы. Заметим, что мы можем использовать a^{2^L} в правой части (2.2.2), даже если при его вычислении произошла ошибка. Несмотря на то, что в правой и левой части находятся произведения, каждый множитель производит проверку своей части вычислений независимо от другого. Если правая и левая части (2.2.2) равны, то можно с высокой степенью уверенности говорить о том, что все вычисления были проведены корректно. Вероятность двух вычислительных ошибок, компенсирующих друг друга, крайне низка, кроме тривиальных случаев вроде общего множителя равного 0.

Количество множителей можно увеличить, проверяя одновременно k вычислений. При этом первые $k - 1$ множителей в левой части будут совпадать с последними $k - 1$ множителями в правой части (рис. 2.1). Обозначим совпадающие множители как d :

$$d = \prod_{i=1}^{k-1} a^{2^{iL}} \bmod m \quad (2.2.3)$$

Тогда проверку Гербица можно сформулировать как

$$d \cdot a^{2^{kL}} \bmod m \stackrel{?}{=} (a \cdot d)^{2^L} \bmod m \quad (2.2.4)$$

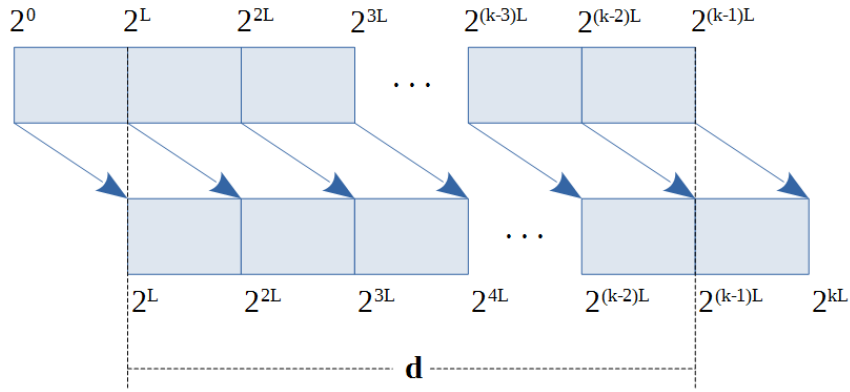


Рис. 2.1: Проверка Гербица

Такой подход позволяет не хранить все промежуточные $a^{2^{iL}}$, а накапливать их в d по мере выполнения вычислений. С точки зрения производительности, нам необходимо выполнить k дополнительных умножений и L возведений в квадрат. Эти операции в данном контексте примерно равнозначны, поэтому при выборе параметров проверки k и L должны быть близки. Гербиц предлагает в качестве k брать L , но на практике часто приходится подгонять их произведение под заданное значение, что можно сделать гораздо точнее, если они различаются.

В случае, если проверка Гербица закончилась неудачно, необходимо начинать вычисления сначала, поскольку нет способа определить в каком конкретно месте случилась ошибка. Это не очень удобно, ведь никто не хочет терять 100% работы. На практике часто используются контрольные точки — в случае неудачной проверки, вычисления откатываются к последней контрольной точке. Это незначительно увеличивает накладные расходы, но гораздо удобнее.

Для примера, возьмём уже упоминавшееся число $281 \cdot 2^{2051865} + 1$. Тест простоты Прота требует проверки следующего выражения:

$$3^{281 \cdot 2^{2051864}} \equiv -1 \pmod{281 \cdot 2^{2051865} + 1}$$

В качестве начального значения возьмём $a = 3^{281}$. Это значение не является большим числом в нашем понимании, его можно вычислить несколькими разными методами умножения для гарантии корректности. Основной же работой является возведение этого значения в квадрат 2051864 раз. Для удобства, установим 10 контрольных точек. Это означает, что нам нужно выполнять проверку Гербица после каждых 205186 возведений в квадрат. Ближайший квадрат равен $L^2 = 452^2 = 204304$. После десяти контрольных точек мы достигнем 2043040 возведений в квадрат, останется “хвостик” из 8824 операций. К этому “хвостику” можно снова применить проверку Гербица, но уже с другими параметрами (и бóльшими накладными расходами). Если же мы возьмём $k = 448$, $L = 458$, то их произведение $L_2 = 448 \cdot 458 = 205184$ даст нам “хвостик” из 24 операций. Накладные расходы без учёта “хвостика” составят $(448 + 458) \cdot 10 = 9060$ операций, или 0,44%.

2.3 Сертификат Атнашева

Однако, проверки Гербица недостаточно, чтобы отказаться от двойной проверки в распределённых вычислениях. Во-первых, решение о прохождении или непрохождении проверки принимается тем же самым вычислительным устройством, что производит вычисления. В случае достаточно серьёзной неисправности, возможен ложноположительный результат. Кроме того, восстановление из контрольной точки также несёт риски, связанные с повреждением данных при копировании (например, при перегреве модуля памяти). Во-вторых, в случае добровольных распределённых вычислений возможен намеренный саботаж со стороны недобросовестных участников. Мотивация может быть самой разной

— от накрутки рейтинга участника до дискредитации администрации конкретного проекта. Нельзя исключать и вероятность ничем не мотивированного цифрового хулиганства. Таким образом, отказ от двойной проверки возможен только при наличии механизма строгого доказательства корректности результата вычислений.

В качестве такого механизма может выступать немного модифицированная проверка Гербица. Модификация заключается в том, что промежуточные значения не накапливаются в $\mathbf{d}$, а сохраняются в виде контрольных точек $\mathbf{r}_i$ и загружаются на доверенный сервер проверки. При этом, при вычислении очередной контрольной точки участник может использовать собственную проверку Гербица (как в примере выше). Чтобы не путать параметры различных проверок, обозначим общее количество контрольных точек как ℓ , а количество возведений в квадрат в одной контрольной точке как M , выбранных так, что ℓM близко к n . Таким образом,

$$\mathbf{r}_i = \mathbf{a}^{2^{iM}} \bmod m \quad (2.3.1)$$

Доверенный сервер может провести проверку Гербица, используя эти контрольные точки в формулах (2.2.3) и (2.2.4), что может подтвердить корректность последней точки $\mathbf{r}_\ell$. При правильном выборе параметров, из $\mathbf{r}_\ell$ можно достаточно быстро получить результат теста простоты и сравнить его с присланным участником, по сути выполнив двойную проверку “хвостика”.

Однако, у этой схемы есть несколько серьёзных недостатков. Прежде всего, это нагрузка на доверенный сервер. В проекте распределённых вычислений могут состоять тысячи участников и десятки тысяч вычислительных устройств. От доверенного сервера может требоваться проверять сотни тысяч тестов в день. Чтобы справиться с такой нагрузкой, необходимо устанавливать как можно меньшее M . Но при уменьшении M возрастает ℓ . А так как мы говорим о больших числах, то каждая контрольная точка имеет существенный размер, вплоть до десятков мегабайт. На практике, максимальное число контрольных точек, загружаемых на сервер каждым участником публичного распределённого проекта, ограничено несколькими десятками. Что, в свою очередь, устанавливает нижнюю границу для M . А так как каждая проверка включает в себя M возведений в квадрат, то администрация проекта распределённых вычислений должна сама обладать существенными вычислительными мощностями, всего лишь в ℓ раз меньшими, чем сумма мощностей всех участников.

Для решения этой проблемы Атнашевым была предложена концепция «сертификата» [8]. Сертификат — это пара больших чисел $(\mathbf{x}, \mathbf{y})$, вычисленных доверенным сервером, где $\mathbf{x}$ является открытой частью, а $\mathbf{y}$ секретной. Любая сторона, будь то администрация или участник, может возвести $\mathbf{x}$ в квадрат M раз и сообщить результат доверенному серверу. Если результат совпадёт с $\mathbf{y}$, то сертификат считается подтверждённым, а тест простоты, связанный с этим сертификатом, доказанно корректным. Таким образом, доверенный сервер может переложить большую часть работы по доказательству корректности на самих участников. При этом, дополнительная нагрузка на участников составит $1/\ell$ от основной работы, что всегда ниже дополнительной нагрузки в 100% при двойной проверке. Кроме того, доверенный сервер может даже не хранить $\mathbf{y}$, а только достаточно длинный отпечаток (хеш) этого числа. В таком случае размер сертификата уменьшится в два раза, а подтверждение сертификата будет заключаться в сравнении двух отпечатков.

Сертификат вычисляется доверенным сервером из загруженных контрольных точек и набора случайных степеней безопасности e_i :

$$\mathbf{r}_0 = \mathbf{a}, \quad e_i = \text{random}(), \quad \mathbf{x} = \prod_{i=0}^{\ell-1} \mathbf{r}_i^{e_i} \bmod m, \quad \mathbf{y} = \prod_{i=1}^{\ell} \mathbf{r}_i^{e_{i-1}} \bmod m \quad (2.3.2)$$

Начальное значение $\mathbf{r}_0$ является основой доказательства, поэтому должно вычисляться доверенным сервером самостоятельно. Степени безопасности e_i выполняют несколько функций. Во-первых, изоли-

руют контрольные точки друг от друга, предотвращая атаки вида

$$\prod_{i=1}^{\ell-1} r_i \equiv 1 \pmod{m}, \quad r_\ell = a^{2^M} \pmod{m} \quad (2.3.3)$$

Злонамеренный участник достаточно легко может сконструировать такие r_i из случайных величин и обратного к их произведению. Случайные e_i делают успех такой атаки практически недостижимым, поскольку разбивают проверку на отдельные независимые проверки

$$r_{i+1}^{e_i} \stackrel{?}{\equiv} (r_i^{e_i})^{2^M} \pmod{m} \quad (2.3.4)$$

В целом, всё аналогично проверке Гербица (рис. 2.2).

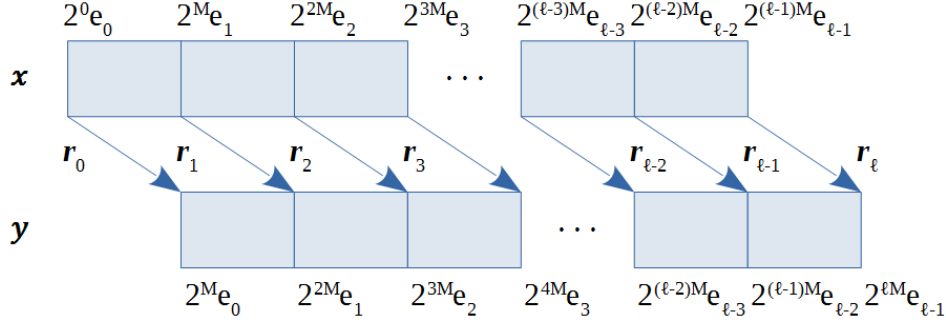


Рис. 2.2: Проверка сертификата

Другой функцией e_i является так называемое “шифрование” сертификата. Дело в том, что в случае, если проверкой сертификата занимаются участники, есть вероятность, что сертификат попадёт к тому же самому участнику, что производил основные вычисления. Если этот участник злонамеренный, либо же просто хочет сэкономить на вычислениях, он может повторить те же самые операции, что произвёл доверенный сервер, и получить y без выполнения M возведений в квадрат. Это возможно только если e_i отсутствуют, либо же участник может легко угадать их значения. Поэтому к генерации e_i следует относиться как к генерации секретного ключа.

Описанный протокол позволяет на практике доказывать корректность произведённых вычислений. Однако целью вычислений является получение результата. В случае теста Ферма возможно два вида результата: равен 1 (число простое) и не равен 1 (число составное). В случае, если результат не равен 1, нас не интересует его конкретное значение, а значит нас не интересуют возможные атаки, которые искажают это значение. В случае же простого числа, мы должны исследовать возможность атаки, меняющей 1 на какое-то другое значение.

К сожалению, такая атака возможна. Например, можно умножить r_ℓ на -1 (если $\ell M = n$), либо на k -ый первообразный корень из единицы. Если $e_{\ell-1}$ делится на k , то при построении сертификата корень из единицы превратится в 1, и сертификат будет успешно подтверждён. Однако, такое искажённое r_ℓ даст в качестве результата теста число, отличное от 1 (например, -1 в простейшем случае). Тестируемое большое число будет ошибочно признано составным. Вдобавок к хулиганству, мотивацией злонамеренного участника в этом случае может послужить желание заполучить простое число только для себя. Эта атака возможна потому, что найти первообразные корни из единицы для простого числа не составляет труда, особенно если это число вида $k \cdot 2^n + 1$ ($\varphi(m) = k \cdot 2^n$). В случае чисел другого вида, можно поискать небольшие делители $\varphi(m)$, ведь чем меньше степень корня из единицы, тем выше шанс на успешную атаку.

Тот же принцип можно использовать для защиты от такой атаки. Доверенный сервер, проверяя результат теста, может возвести его в степени небольших делителей $\varphi(m)$. В случае, если одно из значений обратится в 1, это должно являться как минимум поводом для беспокойства.

2.4 Функция Петшака

Основным недостатком описанного выше протокола является необходимость загрузки больших объёмов данных. Возможность улучшить этот показатель была найдена в неожиданном месте. В качестве замены майнинга для криптовалют, Петшаком была предложена функция гарантированной задержки времени (verifiable delay function, VDF) [9]. Эта функция возводит число в степень 2^n и предоставляет доказательство проделанной работы. Доказательство конструируется способом, похожим на описанный в предыдущем разделе, но есть и важные отличия. Во-первых, функция требует, чтобы ℓ являлось степенью двойки. Во-вторых, e_i вычисляются с помощью детерминированной функции, зависящей от хеша r_ℓ . В-третьих, детерминированность e_i позволяет вычислять частичные произведения контрольных точек на стороне участника, что снижает количество загружаемых на доверенный сервер больших чисел до $\log_2 \ell$.

Функция Петшака представляет собой рекурсивный алгоритм, на каждом шаге которого рассматривается пара больших чисел (x_i, y_i) . Требуется установить корректность следующего выражения:

$$y_i \stackrel{?}{=} x_i^{2^T} \bmod m \quad (2.4.1)$$

Мы можем непосредственно выполнить эту проверку, что потребует T возведений в квадрат. Но заметим, что если умножить каждое из этих чисел на значение, находящееся “посередине” между ними

$$p_i = x_i^{2^{T/2}} \bmod m \quad (2.4.2)$$

то получившаяся пара чисел $(x_i \cdot p_i, p_i \cdot y_i)$ будет удовлетворять (2.4.1) для $T/2$ только тогда, когда исходная пара чисел удовлетворяла (2.4.1). Для дополнительной изоляции множителей, один из них возводится в степень, полученную путём хеширования текущего состояния всех параметров. Получается следующий алгоритм:

Algorithm 2.3: Функция Петшака.

```

1   $x_0 \leftarrow a, \quad y_0 \leftarrow r_\ell, \quad T \leftarrow \ell M$ 
2  for  $i \leftarrow 0$  to  $\log_2 \ell - 1$  do
3       $T \leftarrow T/2$ 
4       $p_i \leftarrow x_i^{2^T} \bmod m$ 
5       $h_i \leftarrow \text{hash}(x_i, p_i, y_i)$ 
6       $x_{i+1} \leftarrow x_i^{h_i} \cdot p_i \bmod m$ 
7       $y_{i+1} \leftarrow p_i^{h_i} \cdot y_i \bmod m$ 
8  end
```

Заметим, что для вычисления p_i нет необходимости возводить x_i в степень. Ведь p_i является произведением 2^i контрольных точек:

$$p_i = \prod_{\substack{j=1 \\ k=j\ell/2^i - \ell/2^{i+1}}}^{2^i} r_k^{e_k} \quad e_k = \prod_{t=0}^{i-1} \text{bit}(2^i - j, i - t - 1) h_t \quad (2.4.3)$$

Рассмотрим пример. Пусть $\ell = 8$, следующая таблица показывает значения, полученные на каждом шаге функции Петшака:

i	T	p_i	x_{i+1}	y_{i+1}
0	$4M$	r_4	$r_0^{h_0} r_4$	$r_4^{h_0} r_8$
1	$2M$	$r_2^{h_0} r_6$	$r_0^{h_0 h_1} r_2^{h_0} r_4^{h_1} r_6$	$r_2^{h_0 h_1} r_4^{h_0} r_6^{h_1} r_8$
2	M	$r_1^{h_0 h_1} r_3^{h_0} r_5^{h_1} r_7$	$r_0^{h_0 h_1 h_2} r_1^{h_0 h_1} r_2^{h_0 h_2} r_3^{h_0} r_4^{h_1 h_2} r_5^{h_1} r_6^{h_2} r_7$	$r_1^{h_0 h_1 h_2} r_2^{h_0 h_1} r_3^{h_0 h_2} r_4^{h_0} r_5^{h_1 h_2} r_6^{h_1} r_7^{h_2} r_8$

Заметим, что каждый $\mathbf{r}_j$ в $\mathbf{x}_{i+1}$ имеет уникальный e_j . При этом операция проверки $(\mathbf{x}_3, \mathbf{y}_3)$ выглядит идентично изображённой на рис. 2.2.

Использование рекурсии предоставляет некоторые дополнительные полезные возможности. В качестве начального значения T можно брать n , даже если оно не делится на M . Это означает, что на каком-то шаге рекурсии T станет нечётным. Чтобы правильно обработать эту ситуацию, в начало шага следует добавить операцию

if $odd(T)$ **then** $\mathbf{x}_i \leftarrow \mathbf{x}_i^2 \bmod m, T \leftarrow T - 1$

Эта операция “поддвигает” $\mathbf{x}_i$ так, чтобы T стало чётным. В результате, контрольные точки, необходимые для вычисления $\mathbf{p}_i$, оказываются расположены через нерегулярные интервалы. Это не сильно усложняет их сохранение, зато пропадает “хвостик”, $\mathbf{r}_\ell$ всегда оказывается равным результату теста.

Доверенный сервер может воспроизвести рекурсию имея только последнюю контрольную точку $\mathbf{r}_\ell$ и все частичные произведения $\mathbf{p}_i$. Если хоть одно $\mathbf{p}_i$ является ошибочным, то есть не лежит “посередине” между $\mathbf{x}_i$ и $\mathbf{y}_i$, то проверка закончится неудачно. В то же время, атака с использованием корней из единицы становится проще, поскольку все e_j вычисляются детерминированно. Злонамеренный участник может потратить время умножая контрольные точки на корень из единицы, пока операция проверки не будет иметь одинаковые степени корня справа и слева. Это делает проверку на такую атаку обязательной на доверенном сервере.

Полученная в конце работы алгоритма пара больших чисел $(\mathbf{x}_{\log_2 \ell}, \mathbf{y}_{\log_2 \ell})$ может выступать в качестве сертификата. Однако, такой сертификат является детерминированным, а значит участникам известна секретная часть сертификата выполненного ими теста, что обесценивает его применение. Для решения этой проблемы Атнашев предлагает использовать дополнительную операцию “шифрования” сертификата:

$$e = \text{random}(), \quad \mathbf{x}' = \mathbf{x}^e \bmod m, \quad \mathbf{y}' = \mathbf{y}^e \bmod m \quad (2.4.4)$$

Секретный ключ e должен быть достаточно длинным, чтобы вычисление дискретного логарифма было невозможно за разумное время, даже если $\varphi(m)$ состоит из небольших делителей.

В то же время, функцию Петшака можно применять и вообще без сертификата. Алгоритм позволяет уменьшать T до таких малых значений, что доверенный сервер может самостоятельно произвести финальную проверку, без необходимости сохранения сертификата. Однако, стоит помнить, что мы работаем с большими числами. Даже несмотря на то, что участникам нужно передавать по сети всего $\log_2 \ell$ чисел, локально они должны хранить все ℓ контрольных точек, поскольку вычисление частичных произведений возможно только после получения последней контрольной точки. И если хранение тысячи контрольных точек по несколько мегабайт каждая в современном мире не составляет труда, то хранение миллиона таких точек уже затруднительно. Это ограничивает глубину рекурсии и делает применение только одной функции Петшака непрактичным. Комбинация функции Петшака с сертификатом Атнашева, с проверкой Гербица каждой контрольной точки, является самым гибким и надёжным инструментом на практике.

2.5 Небинарная степень

Внимательный читатель мог заметить, что все формулы, приведённые в предыдущих разделах, остаются верными, если в показателе степени заменить основание 2 на произвольное основание b . Таким образом, описанные методы проверки можно применять и для показателей вида b^n . Однако, практическая реализация такой замены сталкивается с некоторыми трудностями.

Дело в том, что для проверки корректности вычисления степени вида b^n приходится помимо операций возведения в квадрат также выполнять операции умножения больших чисел, даже если основание степени является небольшой константой $a = 3$. Это может показаться странным, ведь возведение в

степень слева направо позволяет заменить умножение больших чисел на умножение на маленькую константу. Однако вспомним, что все методы проверки основываются на использовании промежуточных значений, полученных последовательным возведением в степень, как в (2.2.1). Например, при возведении в степень $5^4 = 625$ нам нужно сохранить промежуточное значение со степенью $5^2 = 25$ для проверки. Возведение в степень 625 слева направо выполняется за 10 операций:

j	9	8	7	6	5	4	3	2	1	0
e_j	1	0	0	1	1	1	0	0	0	1
$\mathbf{r}$	$\mathbf{a}$	$\mathbf{a}^2$	$\mathbf{a}^4$	$\mathbf{a}^9$	$\mathbf{a}^{19}$	$\mathbf{a}^{39}$	$\mathbf{a}^{78}$	$\mathbf{a}^{156}$	$\mathbf{a}^{312}$	$\mathbf{a}^{625}$

Ни одно из промежуточных значений не имеет степень 25. Более того, возведение в степень 25 имеет совсем другие промежуточные значения:

j	4	3	2	1	0
e_j	1	1	0	0	1
r	a	a^3	a^6	a^{12}	a^{25}

Таким образом, эти операции не могут выполняться одна в другой. Чтобы вычислить $\mathbf{a}^{5^4}$ из $\mathbf{a}^{5^2}$, нужно выполнить операцию возведения основания $\mathbf{a}' = \mathbf{a}^{5^2}$ в степень 5^2 . А это основание уже не является маленькой константой.

В результате, тест простоты Ферма числа вида $b^n + 1$ с проверкой Гербица оказывается на 50% медленнее, чем без проверки. Ситуацию можно значительно улучшить, реализовав один из алгоритмов возведения в степень, минимизирующих количество умножений [6]. Конкретно в нашем случае наиболее подходящим алгоритмом является метод скользящего окна.

Метод скользящего окна длины W заключается в разделении показателя степени e на две части. Одна часть представляет собой степень двойки, другая содержит участи из не более чем W бит, с самым младшим битом не равным нулю. Например, $e = 11957708941720303968251$ разбивается следующим образом:

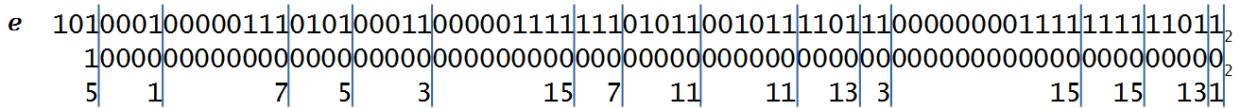


Рис. 2.3: Скользящее окно $W = 4$.

Возведение в степень двойки осуществляется слева направо, а умножение на ненулевые участки выполняется с использованием словаря. Размер словаря равен 2^{W-1} больших чисел, поскольку в нём нужно хранить только нечётные степени. Выбор W зависит от доступного объёма памяти, который может занять словарь, а так же от количества операций, необходимых для построения словаря. Для небольших ϵ нет смысла терять время на построение большого словаря, и наоборот.

Производительность алгоритма оказывается выше простого возведения в степень слева направо за счёт того, что операция умножения выполняется в худшем случае один раз после W возведений в квадрат, а для произвольных степеней в среднем один раз после $W + 1$ возведений в квадрат. В практическом плане, с учётом доступной памяти, для больших чисел W выбирается в диапазоне 5—10. Это означает, что для произвольных степеней количество операций умножения составляет 10-20% от количества возведений в квадрат. Таким образом, при использовании скользящего окна тест простоты Ферма с проверкой Гербица оказывается всего на 10-20% медленнее, чем без проверки.

Algorithm 2.4: Возведение в степень со скользящим окном W .

Output: $r = a^e \bmod m$

```

1   $d_0 \leftarrow a, \quad s \leftarrow a^2 \bmod m$ 
2  for  $i \leftarrow 1$  to  $2^{W-1} - 1$  do
3       $d_i \leftarrow s \cdot d_{i-1} \bmod m$ 
4  end
5   $r \leftarrow 1$ 
6  for  $j \leftarrow \lfloor \log_2 e \rfloor$  to 0 do
7      if  $e_j = 0$  then
8           $r \leftarrow r^2 \bmod m$ 
9      else
10          $i \leftarrow \max(j - W + 1, 0)$ 
11         while  $e_i = 0$  do  $i \leftarrow i + 1$ 
12          $u \leftarrow 0$ 
13         while  $j > i$  do
14             if  $r \neq 1$  then  $r \leftarrow r^2 \bmod m$ 
15              $u \leftarrow 2u + e_j$ 
16              $j \leftarrow j - 1$ 
17         end
18         if  $r = 1$  then  $r \leftarrow d_u$ 
19         else  $r \leftarrow r \cdot d_u \bmod m$ 
20     end
21 end

```

2.6 Обобщение Ли

Применение скользящего окна позволяет использовать строгие проверки и сертификаты для теста Ферма чисел вида $k \cdot b^n + c$. Но такое решение не является идеальным, поскольку приводит как к увеличению количества операций, так и к увеличению обмена данными с медленной памятью из-за необходимости использования словаря. Кроме того, никакие алгоритмы, ускоряющие возведение в степень, не помогут применить строгие проверки к числам, у которых показатель степени в тесте Ферма не может быть представлен в виде b^n с достаточно большим n . Примером таких чисел являются факториальные простые числа $n! \pm 1$. Эти числа представляют интерес, поскольку для них существуют истинные тесты простоты. Но поиск таких чисел затруднён тем, что приходится выполнять двойную проверку.

Для решения всех этих проблем Ли предложил метод обобщения всех описанных выше проверок и сертификатов для произвольных степеней [10]. В основе этого метода лежит независимое рассмотрение операций возведения в квадрат и умножения при возведении в степень слева направо. Допустим, на j -м шаге алгоритма 2.2 мы имеем промежуточное значение

$$r_j = a^{\lfloor e/2^j \rfloor} \bmod m \quad (2.6.1)$$

Тогда на шаге $j - L$ (индекс j уменьшается) это значение будет возведено в квадрат L раз, а также умножено на основание, возведённое в степень, являющуюся комбинацией всех L битов e_j , встретившихся между этими шагами. Иными словами, эта степень является битовой подстрокой числа e длины L , которую мы обозначаем как $e_{[\text{начало}, \text{начало}+L)}$. Промежуточное значение на шаге $j - L$ можно записать как

$$r_{j-L} = r_j^{2^L} \cdot a^{e_{[j-L, j)}} \bmod m \quad (2.6.2)$$

Точно так же можно выразить промежуточное значение на шаге $j - 2L$ через значение на шаге $j - L$:

$$\mathbf{r}_{j-2L} = \mathbf{r}_{j-L}^{2^L} \cdot \mathbf{a}^{e_{[j-2L, j-L]}} \bmod m \quad (2.6.3)$$

Эти два выражения аналогичны выражениям (2.2.1). А значит, мы можем сконструировать выражение, аналогичное (2.2.2), перемножив их:

$$\mathbf{r}_{j-L} \cdot \mathbf{r}_{j-2L} \bmod m \stackrel{?}{=} (\mathbf{r}_j \cdot \mathbf{r}_{j-L})^{2^L} \cdot \mathbf{a}^{e_{[j-L, j]} + e_{[j-2L, j-L]}} \bmod m \quad (2.6.4)$$

Для проверки этого условия необходимо выполнить L возведений в квадрат, а также возвести основание $\mathbf{a}$ в степень суммы двух битовых подстрок длины L , то есть в степень незначительно превышающую 2^L . Если основание $\mathbf{a}$ является маленькой константой, то эта операция сравнима по времени выполнения с L возведений в квадрат. Если нет, то можно применить метод скользящего окна, который незначительно медленнее.

Обобщение (2.6.4) можно применить непосредственно к проверке Гербица (2.2.4). Накопление совпадающих множителей $\mathbf{d}$ производится полностью аналогично:

$$\mathbf{d} = \prod_{i=1}^{k-1} \mathbf{r}_{j-iL} \bmod m \quad (2.6.5)$$

Но кроме этого также накапливается сумма битовых подстрок $\mathbf{s}$:

$$\mathbf{s} = \sum_{i=1}^k e_{[j-iL, j-iL+L]}, \quad \mathbf{s} < k2^L \quad (2.6.6)$$

Наконец, выражение проверки, получившей название проверка Гербица-Ли:

$$\mathbf{d} \cdot \mathbf{r}_{j-kL} \bmod m \stackrel{?}{=} (\mathbf{r}_j \cdot \mathbf{d})^{2^L} \cdot \mathbf{a}^{\mathbf{s}} \bmod m \quad (2.6.7)$$

При практической реализации этой проверки возможно выполнить обе операции возведения в степень одновременно. Для этого достаточно в алгоритме 2.2 задать начальные значения

$$\mathbf{r} = \mathbf{r}_j \cdot \mathbf{d} \cdot \mathbf{a}^{\lfloor \mathbf{s}/2^L \rfloor}, \quad \mathbf{e} = \mathbf{s} \bmod 2^L, \quad j = L - 1 \quad (2.6.8)$$

В конце работы алгоритма, после L возведений в квадрат и домножений на основание, результат будет равен правой части (2.6.7). Таким образом, накладные расходы проверки Гербица-Ли оказываются сравнимы с накладными расходами проверки Гербица.

Выражение (2.6.7) также хорошо показывает главную особенность метода: проверка Гербица-Ли доказывает корректность вычислений от контрольной точки $\mathbf{r}_j$ до контрольной точки $\mathbf{r}_{j-kL}$. Это означает, что в процессе проверки не происходит смены основания $\mathbf{a}$. Если оно было небольшой константой, то и остаётся небольшой константой после проверки. Тест простоты Ферма числа произвольной формы может быть выполнен с проверкой Гербица-Ли почти так же быстро, как без неё.

Рассмотрим пример, возведение в степень $\mathbf{e} = 11957708941720303968251$ с $L = 15$, $k = 5$. Показатель степени может быть представлен как $\mathbf{e} = 10371 \cdot 2^{60} + 21600 \cdot 2^{45} + 32601 \cdot 2^{30} + 15808 \cdot 2^{15} + 8187$. Возьмём $j = 75$, $\mathbf{r}_{75} = 1$. Тогда получаем следующие промежуточные значения:

$$\begin{aligned} \mathbf{r}_{60} &= \mathbf{r}_{75}^{2^{15}} \cdot \mathbf{a}^{10371} \bmod m \\ \mathbf{r}_{45} &= \mathbf{r}_{60}^{2^{15}} \cdot \mathbf{a}^{21600} \bmod m \\ \mathbf{r}_{30} &= \mathbf{r}_{45}^{2^{15}} \cdot \mathbf{a}^{32601} \bmod m \\ \mathbf{r}_{15} &= \mathbf{r}_{30}^{2^{15}} \cdot \mathbf{a}^{15808} \bmod m \\ \mathbf{r}_0 &= \mathbf{r}_{15}^{2^{15}} \cdot \mathbf{a}^{8187} \bmod m \end{aligned}$$

Проверка Гербица-Ли заключается в сравнении

$$r_{60} \cdot r_{45} \cdot r_{30} \cdot r_{15} \cdot r_0 \bmod m \stackrel{?}{=} (r_{75} \cdot r_{60} \cdot r_{45} \cdot r_{30} \cdot r_{15})^{2^{15}} \cdot a^{88567} \bmod m$$

Обобщение Ли точно так же оказывается применимо к функции Петшака. На каждом шаге рекурсии битовое представление показателя степени разделяется на две равные половины, одна половина умножается на хеш и прибавляется к другой. В итоге, s является суммой всех битовых подстрок, положение которых соответствует положению контрольных точек r_k , умноженных на соответствующую степень безопасности e_k .

В случае, если на каком-то шаге рекурсии T оказывается нечётным, то показатель степени разделяется на две части, различающиеся по длине на 1 бит. Но функция Петшака-Ли позволяет полностью избежать подобной ситуации. Заметим, что в предыдущем примере длина показателя составляла 74 бита. Мы добавили ведущий ноль, чтобы получилось 75 бит. Начальная точка при этом оказалась равна 1. Функция Петшака не может работать, если $x_0 = 1$, но функция Петшака-Ли с лёгкостью обрабатывает такую ситуацию. Допустим, мы хотим применить её к предыдущему примеру с $\ell = 4$. Для надёжности, для каждой контрольной точки мы будем использовать проверку Гербица-Ли с $L = 5$, $k = 4$. Таким образом, контрольные точки будут сохраняться через каждые 20 бит, а всего показатель степени будет состоять из $\ell k L = 80$ бит, из которых 6 ведущих будут равны 0. При этом, в первой проверке Гербица-Ли первые два значения будут равны 1.

Сертификат Атнашева-Ли должен включать в себя s , поскольку операцией проверки теперь является

$$y \stackrel{?}{=} x^{2^M} \cdot a^s \bmod m \quad (2.6.9)$$

В то же время, если используется шифрование (возведение обеих частей проверки в секретную степень e), то сертификат должен включать в себя es , что позволяет злонамеренному участнику элементарным образом получить e . Решением является включение в открытую часть сертификата шифрованного основания:

$$a' = a^e \bmod m \quad (2.6.10)$$

При проверке такого сертификата можно так же использовать описанную выше оптимизацию, позволяющую выполнить обе операции возведения в степень одновременно. Так как по требованиям безопасности a' должно быть большим числом, рекомендуется использовать метод скользящего окна, начальные значения которого задаются похожим на (2.6.8) образом.

2.7 Последовательности Люка

Возведение в степень — самая популярная, но далеко не единственная операция, используемая в тестах простоты. Другой популярной и во многом похожей операцией является вычисление последовательностей Люка. Истинные тесты простоты чисел Мерсенна и чисел вида $k \cdot b^n - 1$ используют последовательности Люка, при этом в определённых случаях их производительность не отличается от производительности вероятностного теста Ферма, использующего возведение в степень.

Последовательности Люка представляют собой пару целочисленных рекуррентных последовательностей V_n и U_n с параметрами P и Q , с дискриминантом $D = P^2 - 4Q \neq 0$, удовлетворяющих следующим соотношениям:

$$V_0 = 2, \quad V_1 = P, \quad V_n = PV_{n-1} - QV_{n-2} \quad (2.7.1)$$

$$U_0 = 0, \quad U_1 = 1, \quad U_n = PU_{n-1} - QU_{n-2} \quad (2.7.2)$$

Задавая определённые P и Q , можно получить многие известные последовательности, например числа Фибоначчи, числа Люка, числа Мерсенна или даже многочлены Чебышёва первого и второго рода. Но нас прежде всего интересуют частные случаи, позволяющие вычислять эти последовательности с использованием наименьшего числа операций над большими числами. В этом помогает множество тождеств, включающих в себя члены последовательностей Люка. Например, для последовательности V_n выполняются следующие тождества:

$$V_{2n} = V_n^2 - 2Q^n \quad (2.7.3)$$

$$V_{m+n} = V_m V_n - Q^n V_{m-n} \quad (2.7.4)$$

$$V_{mn} = V_m(P = V_n, Q = Q^n) \quad (2.7.5)$$

Эта последовательность используется в тесте простоты Люка–Лемера для чисел Мерсенна $m = 2^p - 1$. В этом тесте требуется вычислить $V_{2^{p-2}} \pmod{m}$ с $P = 4$ и $Q = 1$. Как мы видим, всё сводится к применению (2.7.3) $p - 2$ раз. При $Q = 1$ эта операция превращается в простое возведение в квадрат и вычитание 2, что можно сделать практически бесплатно в стадии нормализации. Таким образом, в плане операций над большими числами этот тест мало отличается от теста Ферма, где нужно выполнить $p - 1$ возведений в квадрат. Именно из-за подобного сходства мы и рассматриваем последовательности Люка в данной главе.

Так же, как и для возведения в степень, вычисление $V_e \pmod{m}$ усложняется, если e не является степенью двойки. Мы не можем использовать прямой аналог алгоритма возведения в степень слева направо, поскольку для этого нам требуется операция, увеличивающая индекс элемента на 1 (аналог умножения на a). Но как следует из (2.7.1), для вычисления следующего элемента требуется знать значение предыдущего элемента, что невозможно в общем случае. Самым простым решением является использование метода, называемого «лесенка Монтгомери» (Montgomery ladder) [11]. Этот метод похож на возведение в степень слева направо, но на каждом шаге хранится два элемента (V_n, V_{n+1}) . В зависимости от значения бита e_j , с помощью (2.7.3) и (2.7.4) вычисляются либо (V_{2n}, V_{2n+1}) , либо (V_{2n+1}, V_{2n+2}) . При этом сохраняется условие, что индекс второго элемента на единицу больше индекса первого. В конце работы алгоритма первый элемент будет равен V_e .

Algorithm 2.5: Лесенка Монтгомери.

Output: $V = V_e \pmod{m}$

```

1   $V \leftarrow 2$ 
2   $V_{+1} \leftarrow P$ 
3  for  $j \leftarrow \log_2 e$  to 0 do
4      if  $e_j \neq 0$  then
5           $V \leftarrow V_{+1} \cdot V - P \pmod{m}$                                 //  $n' = (n + 1) + n = 2n + 1$ 
6           $V_{+1} \leftarrow V_{+1}^2 - 2 \pmod{m}$                                 //  $n' + 1 = 2(n + 1) = 2n + 2$ 
7      else
8           $V_{+1} \leftarrow V_{+1} \cdot V - P \pmod{m}$                                 //  $n' + 1 = (n + 1) + n = 2n + 1$ 
9           $V \leftarrow V^2 - 2 \pmod{m}$                                         //  $n' = 2n$ 
10     end
11 end
```

На любом шаге алгоритма выполняются одни и те же операции: умножение с вычитанием, возведение в квадрат с вычитанием маленькой константы. Операция умножения с вычитанием может быть реализована как вложенная операция (см. раздел 1.10). Таким образом, производительность алгоритма 2.5 сравнима с производительностью алгоритма возведения в степень справа налево при всех $e_i = 1$. Но если алгоритмы возведения в степень ускоряются при малом количестве ненулевых бит в показателе степени, то время выполнения лесенки Монтгомери остаётся неизменным. Это основной недостаток данного метода.

Использование методов вроде скользящего окна также не представляется возможным, поскольку при вычислении элемента V_{m+n} с помощью (2.7.4) требуется знать элемент V_{m-n} . Такой род операций называется «разностное сложение». Последовательности, где каждый следующий элемент получается с помощью разностного сложения предыдущих элементов, называются «цепочки разностного сложения» (differential addition chains, DAC). Очевидно, что лесенка Монтгомери в процессе работы также создаёт такую цепочку. Например, для $e = 97$ получившаяся цепочка выглядит как

$$\text{ladder}(97) = 0, 1, 1, 2, 3, 4, 6, 7, 12, 13, 24, 25, 48, 49, 97, 98.$$

Здесь и далее мы будем указывать только индексы элементов, образующих цепочку.

Задача нахождения цепочки разностного сложения минимальной длины оказывается весьма нетривиальной. Для её решения разработано множество алгоритмов: цепочки Евклида, Блейхенбахера, Цуруоки, PRAC Монтгомери [12, раздел 4.8]. Мы рассмотрим весьма простую, но дающую хороший результат на практике S-цепочку.

S-цепочка определяется как рекурсивная функция от двух переменных:

$$S(d, e) = \begin{cases} 0, e & d = 0 \\ S(e - d, e) & e/2 < d \\ S(d, e/2), e - d, e & 0 < d < e/4, e \in 2\mathbb{Z} \\ S(d, e - d), e & \end{cases} \quad (2.7.6)$$

В качестве второй переменной берётся требуемое конечное значение, в качестве первой переменной берётся некоторый начальный шаг. Например, для того же $e = 97$ и начального шага $d = 11$ получается цепочка

$$S(11, 97) = 0, 1, 2, 3, 4, 5, 9, 10, 11, 21, 32, 43, 75, 86, 97.$$

Легко видеть, что эта цепочка не даёт никакого преимущества относительно лесенки Монтгомери. Всё дело в выборе начального шага d . Для нахождения цепочки минимальной длины рекомендуется попробовать несколько значений d в районе золотого сечения e . Более сложные алгоритмы, такие как PRAC Монтгомери, для таких d находят цепочки в среднем меньшей длины, чем S-цепочки (рис. 2.4). Однако, если у нас есть возможность для конкретного e перебрать все возможные d и найти оптимальный, то S-цепочка практически никогда не оказывается длиннее PRAC, а нередко оказывается короче. Например, для $e = 97$ оптимальный $d = 21$:

$$S(21, 97) = 0, 1, 2, 3, 5, 8, 13, 21, 34, 55, 76, 97.$$

Иными словами, выбор алгоритма построения цепочки разностного сложения зависит в том числе от возможности заранее найти оптимальные d для всех возможных e . В практических задачах, таких как тест простоты Моррисона или некоторые алгоритмы факторизации, e часто оказывается гладким, то есть является произведением большого количества небольших простых. Используя (2.7.5), мы можем свести вычисление $V_e \pmod{m}$ к последовательному применению цепочек, соответствующих небольшим простым. Таблицу оптимальных d для S-цепочек простых чисел $< 10^7$ можно найти в исходном коде утилиты PRST.

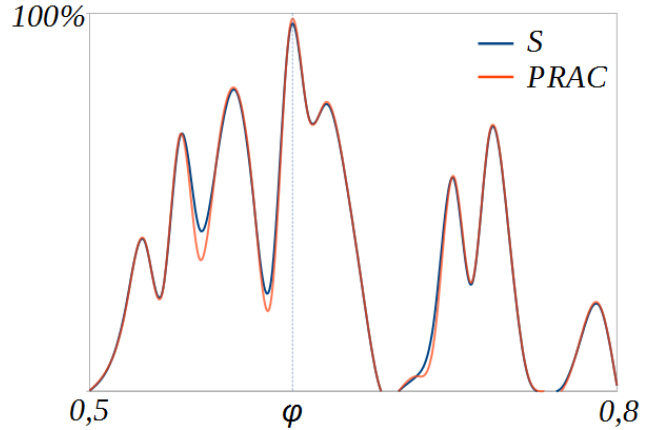


Рис. 2.4: Вероятность оптимального d . PRAC эффективнее в районе золотого сечения.

С точки зрения производительности, применение цепочек разностного сложения позволяет вычислять $V_e \pmod{m}$ используя в среднем $\sim 1,5 \log_2 e$ операций умножения больших чисел, что сравнимо с производительностью алгоритма возведения в степень справа налево для произвольных e . Разумеется, этот результат нельзя улучшить с помощью скользящего окна или оптимизации умножения на небольшие a . Но в случаях, когда производительность вообще не имеет значения, можно получить интересный эффект вычисляя обе последовательности Люка (U_n, V_n) сразу, используя следующие тождества:

$$U_{2n} = U_n V_n \quad (2.7.7)$$

$$2U_{n+1} = V_n + PU_n \quad (2.7.8)$$

$$2V_{n+1} = PV_n + DU_n \quad (2.7.9)$$

$$2U_{m+n} = U_m V_n + V_m U_n \quad (2.7.10)$$

$$2V_{m+n} = V_m V_n + DU_m U_n \quad (2.7.11)$$

Эти тождества дают операции удвоения индекса, увеличения индекса на один, настоящее, не разностное сложение двух пар. А значит, для (U_n, V_n) становятся применимы все алгоритмы, описанные в этой главе. Наконец-то можно использовать метод скользящего окна. А, например, в тесте простоты Моррисона можно использовать проверку Гербица-Ли, что дополнительно увеличивает достоверность этого истинного теста.

2.8 Эллиптические кривые Эдвардса

И возведение в степень по модулю большого числа, и вычисление последовательностей Люка по модулю большого числа интересны прежде всего тем, что при повторении этих операций много раз, значения начинают повторяться (образуют циклическую группу). Анализируя период повторения (порядок группы), можно сделать некоторые выводы о природе большого числа. На этом основываются многие тесты простоты и методы разложения на простые. Но это далеко не единственные операции, способные образовывать циклические группы. Существует множество других математических объектов и операций, которые можно использовать в алгоритмах, анализирующих порядок групп. Иногда один и тот же алгоритм можно запускать несколько раз, используя разные операции и разные группы, пока не найдётся циклическая группа, решающая поставленную задачу.

Примером таких математических объектов являются алгебраические группы. Это множества, состоящие из наборов чисел, удовлетворяющих некоторому алгебраическому уравнению, например $y = kx + b$. Для того, чтобы это множество было группой, должна быть определена операция “сложения”, в том же смысле, в каком для последовательностей Люка определена операция сложения индексов. Если такая операция определена, и обладает всеми нужными свойствами, то мы получаем циклическую группу. Раздел математики, изучающий алгебраические группы, называется «алгебраическая геометрия», поэтому для описания таких групп часто используется геометрическая терминология. Разумеется, если мы выполняем все операции по модулю большого числа, то всякий геометрический смысл теряется, но терминология остаётся прежней.

Упомянутая “линия” $y = kx + b \pmod{m}$ с простым векторным сложением является циклической группой, но не очень интересной. Её порядок всегда равен m . Более интересными группами являются так называемые «эллиптические кривые». Это сложные объекты, используемые во многих разделах математики. Они могут описываться различными уравнениями третьего порядка, которые с помощью несложных преобразований можно привести к нормальной форме Вейерштрасса

$$y^2 = x^3 + ax + b \quad (2.8.1)$$

Операция сложения двух различных элементов $(x_1, y_1) + (x_2, y_2) = (x_3, y_3)$ определяется как

$$\begin{aligned}\lambda &= \frac{y_2 - y_1}{x_2 - x_1}, \\ x_3 &= \lambda^2 - x_1 - x_2, \\ y_3 &= \lambda(x_1 - x_3) - y_1.\end{aligned}\tag{2.8.2}$$

Для удвоения элемента $2(x_1, y_1) = (x_3, y_3)$ определена отдельная операция:

$$\begin{aligned}\lambda &= \frac{3x_1^2 + a}{2y_1}, \\ x_3 &= \lambda^2 - 2x_1, \\ y_3 &= \lambda(x_1 - x_3) - y_1.\end{aligned}\tag{2.8.3}$$

Этих операций достаточно, чтобы определить операцию “умножения” элемента на произвольное натуральное число. Такое умножение можно реализовать, например, с помощью метода скользящего окна. Однако, заметим, что деление по модулю, реализуемое с помощью расширенного алгоритма Евклида, является весьма медленной операцией, тем более для больших чисел. Решением является использование “проективных” координат (X, Y, Z) , определённых как

$$\begin{aligned}(X, Y, Z) : Y^2 &\equiv X^3 + aXZ^4 + bZ^6 \pmod{m}, \\ x &= \frac{X}{Z^2} \pmod{m}, \quad y = \frac{Y}{Z^3} \pmod{m}.\end{aligned}\tag{2.8.4}$$

Относительно несложно переписать операции сложения и удвоения элементов в этих координатах. В результате, для операции сложения элементов потребуется 16 умножений и ни одного деления. Используя метод скользящего окна, можно относительно быстро получить результирующий элемент и вычислить $Z^{-1} \pmod{m}$ только для него.

Тем не менее, производительность такого умножения оставляет желать лучшего. Однако, вспомним, что эллиптические кривые могут задаваться различными уравнениями, не только в форме Вейерштрасса. Для других форм операции над элементами будут также иметь другой вид, возможно более удобный для вычислений. Кроме того, в некоторых случаях сами операции могут быть упрощены. Например, Монтгомери заметил, что в алгоритмах поиска порядка группы используется только одна координата x . А значит, нас интересует такая форма эллиптических кривых, которая позволяет выполнять операции сложения и удвоения используя только одну координату, притом максимально эффективно. Им была предложена следующая форма [13]:

$$By^2 = x^3 + Ax^2 + x\tag{2.8.5}$$

Проективные координаты (X, Z) определяются как

$$\begin{aligned}((X, Z), y) : By^2Z^4 &\equiv X^3Z + AX^2Z^2 + XZ^3 \pmod{m}, \\ x &= \frac{X}{Z} \pmod{m}.\end{aligned}\tag{2.8.6}$$

Монтгомери также была предложена операция однокоординатного удвоения $2(X, Z)$, требующая 5 умножений, и операция однокоординатного сложения $(X_1, Z_1) + (X_2, Z_2)$, требующая 6 умножений и знание разности $(X_1, Z_1) - (X_2, Z_2)$. Иными словами, эта операция является разностным сложением. Для реализации операции умножения элемента на натуральное число можно использовать все методы, которые мы обсуждали в предыдущем разделе, такие как лесенка Монтгомери или цепочки разностного сложения, например PRAC Монтгомери. Такая форма эллиптических кривых и однокоординатные операции над элементами получили название «кривые Монтгомери».

В течение десятилетий кривые Монтгомери были самыми быстрыми и эффективными, пока не появилась новая форма. Она была предложена Эдвардсом, и улучшена Бернштейном и Ланге [14].

$$ax^2 + y^2 = 1 + dx^2y^2 \quad (2.8.7)$$

При $a = 1$ эта форма называется «кривые Эдвардса», при $a \neq 1$ называется «перевитые кривые Эдвардса». Каждая перевитая кривая Эдвардса является бирационально эквивалентной какой-то кривой Монтгомери, и наоборот. Иными словами, существуют простые формулы, позволяющие преобразовывать координаты из одной формы в другую. Соответственно, однокоординатные операции Монтгомери могут быть реализованы и над кривыми Эдвардса. Помимо них, существуют и весьма эффективные двухкоординатные операции. Привлекательность кривых Эдвардса как раз и заключается в том, что можно использовать и те, и другие операции одновременно.

Координаты (X_M, Z_M) на кривой Монтгомери могут быть преобразованы в координаты (Y_{Ed}, Z_{Ed}) на перевитой кривой Эдвардса следующим образом:

$$\begin{aligned} Y_{Ed} &= X_M - Z_M, \\ Z_{Ed} &= X_M + Z_M, \\ a/(a - d) &= (A + 2)/4. \end{aligned} \quad (2.8.8)$$

Однокоординатное разностное сложение на перевитой кривой Эдвардса $(Y_1, Z_1) + (Y_2, Z_2) = (Y_3, Z_3)$ с разностью $(Y_1, Z_1) - (Y_2, Z_2) = (Y_0, Z_0)$ определяется как

$$\begin{aligned} X_M &= (Z_0 - Y_0)(Y_1 Z_2 + Z_1 Y_2)^2, \\ Z_M &= (Z_0 + Y_0)(Y_1 Z_2 - Z_1 Y_2)^2, \\ Y_3 &= X_M - Z_M, \\ Z_3 &= X_M + Z_M. \end{aligned} \quad (2.8.9)$$

Как мы видим, вычисление координат Монтгомери является необходимым промежуточным шагом. То же самое для однокоординатного удвоения $2(Y_1, Z_1) = (Y_3, Z_3)$:

$$\begin{aligned} X_M &= Z_1^2(Y_1^2 - \frac{d}{a}Y_1^2), \\ Y_M &= (Z_1^2 - Y_1^2)(Z_1^2 - \frac{d}{a}Y_1^2), \\ Y_3 &= X_M - Z_M, \\ Z_3 &= X_M + Z_M. \end{aligned} \quad (2.8.10)$$

Эти операции идентичны операциям на кривых Монтгомери и требуют такого же количества умножений.

Для двухкоординатных операций удобно использовать так называемые «расширенные» координаты [15], отличающиеся от проективных наличием дополнительного значения $T = xyZ$. В расширенных координатах уравнение кривой становится совсем простым:

$$\begin{aligned} (X, Y, Z, T) : aX^2 + Y^2 &\equiv Z^2 + dT^2 \pmod{m}, \\ x = \frac{X}{Z} \pmod{m}, \quad y = \frac{Y}{Z} \pmod{m}, \quad T &= \frac{XY}{Z} \pmod{m}. \end{aligned} \quad (2.8.11)$$

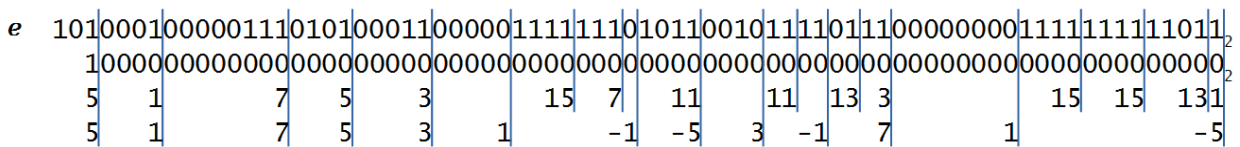
Сложение на перевитой кривой Эдвардса $(X_1, Y_1, Z_1, T_1) + (X_2, Y_2, Z_2, T_2) = (X_3, Y_3, Z_3, T_3)$ определяется как

$$\begin{aligned} X_3 &= (X_1 Y_2 - Y_1 X_2)(T_1 Z_2 + Z_1 T_2), \\ Y_3 &= (Y_1 Y_2 + a X_1 X_2)(T_1 Z_2 - Z_1 T_2), \\ Z_3 &= (Y_1 Y_2 + a X_1 X_2)(X_1 Y_2 - Y_1 X_2), \\ T_3 &= (T_1 Z_2 + Z_1 T_2)(T_1 Z_2 - Z_1 T_2). \end{aligned} \quad (2.8.12)$$

Заметим, что каждая формула состоит из произведения двух значений, всего 4 различных значения используется во всех формулах. Часть этих значений можно вычислить с использованием вложенных операций (см. раздел 1.10). Кроме того, при $a = 1$ формулы становятся полностью свободными от параметров кривой, что очень удобно.

$$\begin{aligned} X_3 &= 2X_1Y_1(2Z_1^2 - Y_1^2 - aX_1^2), \\ Y_3 &= (Y_1^2 + aX_1^2)(Y_1^2 - aX_1^2), \\ Z_3 &= (Y_1^2 + aX_1^2)(2Z_1^2 - Y_1^2 - aX_1^2), \\ T_3 &= 2X_1Y_1(Y_1^2 - aX_1^2). \end{aligned} \tag{2.8.13}$$

Преимущество двухкоординатных операций состоит в том, что наличие настоящего сложения позволяет выполнять операцию умножения элемента на произвольное натуральное число, используя все доступные методы, например уже упомянутый метод скользящего окна. Более того, существует ещё одна полезная возможность. Если в (2.8.12) изменить все знаки на противоположные, то получится операция вычитания $(X_1, Y_1, Z_1, T_1) - (X_2, Y_2, Z_2, T_2) = (X_3, Y_3, Z_3, T_3)$. Таким образом, в отличие от возведения в степень, где вычитание показателя (деление) является дорогостоящей операцией, вычитание на кривых Эдвардса выполняется за точно такое же время, как и сложение. Это позволяет использовать для умножения несмежную форму числа (non-adjacent form, NAF).



Несмежная форма NAF_W может быть получена с помощью следующего алгоритма:

Algorithm 2.6: Несмежная форма ширины W числа e .

Output: $E_i : e = \sum E_i 2^i$

```

1   $i \leftarrow 0$ 
2  while  $e > 0$  do
3      if  $\text{odd}(e)$  then
4           $E_i \leftarrow e \bmod 2^W$ 
5          if  $E_i \geq 2^{W-1}$  then  $E_i \leftarrow E_i - 2^W$ 
6           $e \leftarrow e - E_i$ 
7      else
8           $E_i \leftarrow 0$ 
9       $e \leftarrow e/2$ 
10      $i \leftarrow i + 1$ 
11 end
```

Полученное представление E_i может быть использовано в алгоритме, аналогичном возведению в степень слева направо, только дополненным словарём. Размер словаря оказывается в два раза меньше, чем в методе скользящего окна. Это значит, что при тех же вычислительных затратах, W может быть на 1 больше. Кроме того, словарь может быть нормализован, то есть координаты каждого элемента поделены на Z . При использовании соответствующего алгоритма, это можно сделать, выполнив расширенный алгоритм Евклида только один раз. Нормализованный словарь позволяет сэкономить одно умножение в (2.8.12). А поскольку после каждого сложения (кроме последнего) обязательно идут удвоения, вычислять T_3 в (2.8.12) не требуется. При $a = 1$ и $W = 10$ –15 операция умножения элемента на произвольное натуральное число может быть выполнена с использованием $\sim 7,5$ –7,8 умножений на бит, что значительно лучше $\sim 1,5 \cdot 6 = 9$ умножений на бит при использовании кривых Монтгомери и цепочек разностного сложения.

Операции на кривых Эдвардса позволяют использовать несколько разных методов проверки на ошибки. Например, можно использовать тот факт, что однокоординатные и двухкоординатные операции дают один и тот же результат (после нормализации). Допустим, некоторый алгоритм требует вычисления большого количества элементов через равные промежутки. Начальное значение может быть вычислено с помощью двухкоординатных операций и несмежной формы, после чего элементы, расположенные через равные промежутки, вычисляются с помощью однокоординатного разностного сложения. Последний такой элемент проверяется с помощью снова двухкоординатных операций и несмежной формы.

Также, для двухкоординатных операций в качестве проверки на ошибки можно использовать саму формулу эллиптической кривой

$$aX^2 + Y^2 \stackrel{?}{=} Z^2 + dT^2 \pmod{m}. \quad (2.8.14)$$

Однако, для этого нужно хранить параметры a и d . При известном a , например $a = 1$, есть возможность относительно просто вычислить

$$d = \frac{aX^2 + Y^2 - Z^2}{T^2} \pmod{m} \quad (2.8.15)$$

и сравнить его с оригинальным значением. Для проверки можно даже не хранить значение d , а только его хеш. В случае арифметической ошибки, с большой вероятностью элемент “перескочит” на кривую с другими параметрами, а значит изменится как d , так и хеш. (2.8.15) выполняет роль отпечатка конкретной кривой.

Кроме того, так как двухкоординатные операции на кривых Эдвардса ведут себя аналогично умножению чисел и возведению в квадрат, все алгоритмы проверки, описанные в этой главе, могут быть реализованы и с использованием эллиптических кривых.

Глава 3

Разложение на простые

Раскладывать большие числа на простые множители очень трудно.

Благодарности

Автор выражает благодарность:

Иву Галло / Yves Gallot за то, что открыл мне глаза на алгебраическую сторону БПФ.

Джорджу Уолтману / George Woltman за ответы на мои бесконечные вопросы.

Даррену Ли / Darren Li за советы по существу и редактирование английского перевода.

Роману Трунову за поддержку практической реализации.

сообществу **PrimeGrid** за доверие.

моей семье за безоговорочную поддержку.

спонсорам за пожертвования Монеро на 453MFNnC7N7NK9iK99FoNh3xZ4wY9yqt1Q
k6F62iUZjg6bv6xD3oH3U2EABnUQMmco5BEK5FAhtKF18Mn43QQsysNHX9wB3.

Литература

- [1] D. H. Bailey. FFTs in External or Hierarchical Memory. *Supercomputing '89: Proceedings of the 1989 ACM/IEEE conference on Supercomputing*, pages 234–242, 1989. <https://doi.org/10.1145/76263.76288>.
- [2] Daniel J. Bernstein. Removing redundancy in high-precision Newton iteration, 1998. <https://cr.yp.to/papers.html#fastnewton>.
- [3] Richard Crandall and Barry Fagin. Discrete weighted transforms and large-integer arithmetic. *Mathematics of Computation*, 62, pages 305–324, 1994. <https://doi.org/10.1090/S0025-5718-1994-1185244-1>.
- [4] Colin Percival. Rapid multiplication modulo the sum and difference of highly composite numbers. *Mathematics of Computation*, 72, pages 387–395, 2003. <https://doi.org/10.1090/S0025-5718-02-01419-9>.
- [5] Pavel Atnashev and George Woltman. Edwards curves and FFT-based multiplication. Cryptology ePrint Archive, Report 2021/1061, 2021. <https://eprint.iacr.org/2021/1061>.
- [6] Christophe Doche. Exponentiation. *Handbook of Elliptic and Hyperelliptic Curve Cryptography*, pages 145–168, 2006. <http://hyperelliptic.org/HEHCC/chapters/chap09.pdf>.
- [7] Robert Gerbicz. Fast and robust error checking on Proth/Pepin tests, 2017. <https://www.mersenneforum.org/showthread.php?t=22510>.
- [8] Pavel Atnashev. Efficient Proth/PRP Test Proof Scheme, 2020. <https://www.mersenneforum.org/showthread.php?t=25323>.
- [9] Krzysztof Pietrzak. Simple Verifiable Delay Functions. Cryptology ePrint Archive, Report 2018/627, 2018. <https://eprint.iacr.org/2018/627>.
- [10] Darren Li and Yves Gallot. An Efficient Modular Exponentiation Proof Scheme. arXiv:2209.15623 [cs.CR], 2023. <https://arxiv.org/abs/2209.15623>.
- [11] Peter L. Montgomery. Evaluation of $V_n(P, 1)$ via Lucas chains, 1983. <https://cr.yp.to/bib/1992/montgomery-lucas.pdf>.
- [12] Daniel J. Bernstein and Tanja Lange. Montgomery curves and the Montgomery ladder. Cryptology ePrint Archive, Report 2017/293, 2017. <https://eprint.iacr.org/2017/293>.
- [13] Peter L. Montgomery. Speeding the Pollard and elliptic curve methods of factorization. *Mathematics of Computation*, 48, pages 243–264, 1987. <https://doi.org/10.1090/S0025-5718-1987-0866113-7>.
- [14] Daniel J. Bernstein and Tanja Lange. Faster addition and doubling on elliptic curves. Cryptology ePrint Archive, Report 2007/286, 2007. <https://eprint.iacr.org/2007/286>.

- [15] Huseyin Hisil, Kenneth Koon-Ho Wong, Gary Carter, and Ed Dawson. Twisted Edwards Curves Revisited. Cryptology ePrint Archive, Report 2008/522, 2008. <https://eprint.iacr.org/2008/522>.